BUET_Asymptotes

Bangladesh University of Engineering and Technology

Contents

1 Data structures 2
2 Dynamic Programming 6
3 Number theory 7
4 Combinatorial 9
5 Numerical 10
6 Graph 14
7 Geometry 20
8 Strings 22
9 Various 25

template.cpp

```cpp
#include<bits/stdc++.h>
using namespace std;
#define int long long
#ifdef LOCAL
#include "debug.h"
#else
#define deb(...)
#endif
void solve(){}
int32_t main() {
    ios_base::sync_with_stdio(false); cin.tie(nullptr);
    int t; cin >> t; while(t--)
    solve();
}
```

c

```
#!/bin/bash
if ! g++ -std=c++23 -DLOCAL -Wshadow -Wall -g \
 -fsanitize=address,undefined -D_GLIBCXX_DEBUG \
 -fmax-errors=1 code.cpp -o a
then
    exit 1
fi
echo Compiled!
./a
```

run

```
#!/usr/bin/env bash
for inp in in in[0-9]*; do
    if [ -f $inp ]; then
        echo
        echo "Input: $inp"
        cat $inp
        echo Output:
        ./a.out < $inp
        echo
    fi
done
```

debug.h

```cpp
template<class t>
auto pr(t x) -> decltype(cerr<<x, void()) {cerr<<x;}
void pr(string s) {cerr<<s;}
template<class t>
auto pr(t v) -> decltype(v.begin(), void());
template<class a, class b>
void pr(pair<a,b> p){
  cerr << "{";  pr(p.F);  cerr << ", ";  pr(p.S);  cerr << "}";
}
template<class... a>
void pr(tuple<a...> t){
    cerr << "(";
    apply( [&] (auto... x) {
        ( ( pr(x), cerr << ", " ),  ...);
    },  t);
    cerr << ")";
}
template<class t>
auto pr(t v) -> decltype(v.begin(), void()){
    cerr << "[";
    for(auto x : v){
        pr(x);  cerr << ", ";
    }
    cerr << "]";
}
template<class... t>
void _deb(const t&... x){
    ( ( pr(x), cerr << ", " ),  ...);
    cerr << '\n';
}
#define deb(...) cerr << __LINE__ << "| [" << #__VA_ARGS__ << \
 "] = " ,  _deb(__VA_ARGS__)
```

macros.cpp

```cpp
// *st.find_by_order(index) = value at index
// st.order_of_key(x) = no. of elements strictly less than x
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;
#define ordered_set tree<int, null_type,less<int>, \
 rb_tree_tag,tree_order_statistics_node_update>

#define all(v) v.begin(),v.end()
#define range(v, i, j) v.begin()+i, v.begin()+j+1
#define rep(i, a, b) for(ll i = (a); i < (b); ++(i))
#define sz(x) (ll)(x.size())

#define isSet(x, i) ((x>>i)&1)
#define setbit(x, i) (x | (1LL<<i))
#define resetbit(x, i) (x & (~(1LL << i)))
#define toggleBit(x, i) ((x) ^ (1LL << (i)))
#define clz(x) __builtin_clzll(x)
#define ctz(x) __builtin_ctzll(x)
#define csb(x) __builtin_popcountll(x)
#define msb(x) ((x) ? (63 - __builtin_clzll((ll)(x))) : -1)
#define lsb(x) ((x) ? (__builtin_ctzll((ll)(x))) : -1)
```

gen.cpp

```cpp
mt19937_64 \
 rng(chrono::steady_clock::now().time_since_epoch().count());
#define rnd(l, r) uniform_int_distribution<ll>(l, r)(rng)
// [l,r]
```

brute.cpp

```cpp
// Write brute force for stress testing
```

stress.sh

```
set -e
g++ gen.cpp -o gen
g++ brute.cpp -o bru
for((i = 1; ; ++i)); do
    ./gen $i > tcase
    ./a.out < tcase > myans
    ./bru < tcase > corans
    diff -Z myans corans > /dev/null || break
    echo "Passed test: " $i
done
echo "WA on test:"
cat tcase
echo "Your answer:"
cat myans
echo "Correct answer:"
cat corans
```

mint.h

```cpp
#include "euclid.h"
struct mint{
 ll x; mint() { x=0; }
 mint(ll xx) {x = xx%mod; if(x<0)x+=mod; }
 mint& operator+=(mint b){ x=(x + b.x)%mod; return *this; }
 mint& operator*=(mint b){ x=(x * b.x)%mod; return *this; }
 mint& operator-=(mint b){ x=(x - b.x +mod)%mod; return *this;}
 mint& operator/=(mint b){ return *this *= b.inv(); }
 friend mint operator+(mint a,mint b){ return a+=b; }
 friend mint operator-(mint a,mint b){ return a-=b; }
 friend mint operator*(mint a,mint b){ return a*=b; }
 friend mint operator/(mint a,mint b){ return a/=b; }
 mint operator-() { return mint()-*this; }
 mint inv() { return power(mod-2); } // prime mod
//  mint inv() { // any mod
//    ll x, y, g = euclid(a.x, mod, x, y);
//    assert(g == 1); return mint((x + mod) % mod);
//  }
 mint power(ll n) {
  mint a=*this,  res=1;
  while(n) { if(n&1)res*=a;  a*=a;  n>>=1;}
  return res;
 }
  friend ostream& operator<<(ostream& os, const mint& m) \
   { return os << m.x; }
};

const int N=2000005; mint fact[N],inv_fact[N];
mint power(mint a,ll n){ return a.power(n); }
void prep_factorials(){
 fact[0] = 1;
 for(int i=1; i<N; ++i) fact[i] = fact[i-1] * i;
 inv_fact[N-1] = fact[N-1].inv();
 for(int i=N-2; i>=0; --i) inv_fact[i] = inv_fact[i+1]*(i+1);
}
// for big n and small r, use loop instead
mint ncr(int n,int r){ // limitation : small n
 if(r<0 || r>n) return mint();
 return fact[n] * inv_fact[r] * inv_fact[n-r];
}
mint ncr_lucas(int n, int r) { // limitation : small mod
    if (r < 0 || r > n) return 0;
    mint res(1);
    while (n > 0 || r > 0) {
        int ni = n % mod, ri = r % mod;
        if (ri > ni) return 0;
```

```
        res *= ncr(ni, ri);
        n /= mod; r /= mod;
    }
    return res;
}
```

0.1 Trigonometry

$$\tan(v+w) = \frac{\tan v + \tan w}{1 - \tan v \tan w}$$

$$\sin v + \sin w = 2\sin\frac{v+w}{2}\cos\frac{v-w}{2}$$

$$\cos v + \cos w = 2\cos\frac{v+w}{2}\cos\frac{v-w}{2}$$

$$(V+W)\tan(v-w)/2 = (V-W)\tan(v+w)/2$$

where V, W are lengths of sides opposite angles v, w.

$$a\cos x + b\sin x = r\cos(x-\phi)$$
$$a\sin x + b\cos x = r\sin(x+\phi)$$

where $r = \sqrt{a^2+b^2}, \phi = \text{atan2}(b, a)$.

0.2 Recurrences

If $a_n = c_1 a_{n-1} + \cdots + c_k a_{n-k}$, and $r_1, \ldots, r_k$ are distinct roots of $x^k - c_1 x^{k-1} - \cdots - c_k$, there are $d_1, \ldots, d_k$ s.t.

$$a_n = d_1 r_1^n + \cdots + d_k r_k^n.$$

Non-distinct roots r become polynomial factors, e.g. $a_n = (d_1 n + d_2) r^n$.

0.3 Graphs

For a planar graph with C connected components, V vertices, E edges, and F faces,

$$V - E + F = C + 1.$$

0.4 Geometry

0.4.1 Triangles

Side lengths: a, b, c

Semiperimeter: $p = \dfrac{a+b+c}{2}$

Area: $A = \sqrt{p(p-a)(p-b)(p-c)}$

Circumradius: $R = \dfrac{abc}{4A}$

Inradius: $r = \dfrac{A}{p}$

Length of median (divides triangle into two equal-area triangles): $m_a = \frac{1}{2}\sqrt{2b^2+2c^2-a^2}$

Length of bisector (divides angles in two): $s_a = \sqrt{bc\left[1 - \left(\dfrac{a}{b+c}\right)^2\right]}$

Law of sines: $\dfrac{\sin\alpha}{a} = \dfrac{\sin\beta}{b} = \dfrac{\sin\gamma}{c} = \dfrac{1}{2R}$

Law of cosines: $a^2 = b^2 + c^2 - 2bc\cos\alpha$

Law of tangents: $\dfrac{a+b}{a-b} = \dfrac{\tan\frac{\alpha+\beta}{2}}{\tan\frac{\alpha-\beta}{2}}$

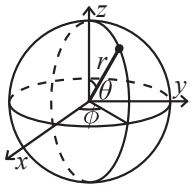
0.4.2 Quadrilaterals

With side lengths a, b, c, d, diagonals e, f, diagonals angle θ, area A and magic flux $F = b^2 + d^2 - a^2 - c^2$:

$$4A = 2ef \cdot \sin\theta = F\tan\theta = \sqrt{4e^2f^2 - F^2}$$

For cyclic quadrilaterals the sum of opposite angles is $180°$, $ef = ac + bd$, and $A = \sqrt{(p-a)(p-b)(p-c)(p-d)}$.

0.4.3 Spherical coordinates



$$x = r\sin\theta\cos\phi \qquad r = \sqrt{x^2+y^2+z^2}$$
$$y = r\sin\theta\sin\phi \qquad \theta = \text{acos}(z/\sqrt{x^2+y^2+z^2})$$
$$z = r\cos\theta \qquad \phi = \text{atan2}(y, x)$$

0.5 Sums

$$c^a + c^{a+1} + \cdots + c^b = \frac{c^{b+1} - c^a}{c-1}, c \neq 1$$

$$1^2 + 2^2 + 3^2 + \cdots + n^2 = \frac{n(2n+1)(n+1)}{6}$$

$$1^3 + 2^3 + 3^3 + \cdots + n^3 = \frac{n^2(n+1)^2}{4}$$

$$1^4 + 2^4 + 3^4 + \cdots + n^4 = \frac{n(n+1)(2n+1)(3n^2+3n-1)}{30}$$

0.6 Series

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \ldots, (-\infty < x < \infty)$$

$$\ln(1+x) = x - \frac{x^2}{2} + \frac{x^3}{3} - \frac{x^4}{4} + \ldots, (-1 < x \leq 1)$$

$$\sqrt{1+x} = 1 + \frac{x}{2} - \frac{x^2}{8} + \frac{2x^3}{32} - \frac{5x^4}{128} + \ldots, (-1 \leq x \leq 1)$$

$$\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \ldots, (-\infty < x < \infty)$$

$$\cos x = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \ldots, (-\infty < x < \infty)$$

Data structures (1)

PersistentSegmentTree.h

```
struct PersistentSegTree {
  struct Node {
    int l = 0, r = 0; ll mx = -inf, cnt = 0;
  };
  vector<Node> st;
  PersistentSegTree() {st.push_back(Node());} // 0 = null

  int update(int prev, int tl, int tr, int pos, ll val = 1) {
    st.push_back(st[prev]); // new copy of prev's root
    int cur = (int)st.size() - 1;
    if (tl == tr) {
      st[cur].mx = val;
      st[cur].cnt += val;
      return cur;
    }
    int tm = (tl + tr) >> 1; // either l/r of root changes
    if (pos <= tm)
        st[cur].l = update(st[prev].l, tl, tm, pos, val);
    else st[cur].r = update(st[prev].r, tm + 1, tr, pos, val);
    st[cur].mx = max(st[st[cur].l].mx, st[st[cur].r].mx);
    st[cur].cnt = st[st[cur].l].cnt + st[st[cur].r].cnt;
    return cur;
  }

  // query(root[i], 0, n-1, l, r) means
  // maximum on range [l,r] in version i
  ll query(int node, int tl, int tr, int l, int r) {
    if (!node || r < tl || tr < l) return -INF;
    if (l <= tl && tr <= r) return st[node].mx;
    int tm = (tl + tr) >> 1;
    return max(
      query(st[node].l, tl, tm, l, r),
      query(st[node].r, tm + 1, tr, l, r)
    );
  }

/* EXAMPLE */
// kth smallest in [L,R], using (root before L) and (root at R)
// call : pst.findkth(root[L - 1], root[R], lo, hi, k);
  int findkth(int rootL, int rootR, int tl, int tr, ll k) {
    if (k < 1 || k > st[rootR].cnt - st[rootL].cnt) return -1;
    if (tl == tr) return tl;
    int tm = (tl + tr) >> 1;
    ll left = st[st[rootR].l].cnt - st[st[rootL].l].cnt;
    if (k <= left)
      return findkth(st[rootL].l, st[rootR].l, tl, tm, k);
    return
    findkth(st[rootL].r, st[rootR].r, tm + 1, tr, k - left);
  }

  /* CUSTOM */
  // Collect all ids in this version whose stored v >= need.
  // Problem statement bounds total leaves accessed, not costly
  void collect(int node, int tl, int tr, ll v_need, vi& out) {
    if (node == 0 || st[node].mx < v_need) return;
    if (tl == tr) { out.push_back(tl); return; }
    int tm = (tl + tr) >> 1;
    collect(st[node].l, tl, tm, v_need, out);
    collect(st[node].r, tm + 1, tr, v_need, out);
  }
};
```

```
/* USAGE */
PersistentSegTree pst;
vector<int> root(N + 1, 0);
for (int i = N - 1; i >= 0; i--) {//or forward, whatever works
  root[i] = pst.update(root[i + 1], 0, N - 1, pos, val);
}
/*
  k-th on tree path : versions for each tree node.
  Each node extends from it's parent's version.
  seg tree is ON the node values (compressed).
  each segtree index HOLDS total count over all it's ancestors
  inner nodes hold total of left child + right child.
  walk down 3 versions simultaneously.
  at current segtree node, for both children :
  use LCA formula to calculate count of elements in that range
  go left or right acccording to current k
*/
```

LazySegmentTree.h

```
using T = ll;
struct Lseg {
    struct node { T mn=0, mx=0, sm=0; }; // remove/add

    int n;
    vector<node> tr;
    vector<T> s, a; // s:set tag, a:add tag

    Lseg(int _n) : n(_n), tr(4*n), s(4*n, inf), a(4*n, 0) {}

    inline node f(const node& i, const node& j) { // merge
        return {min(i.mn, j.mn), max(i.mx, j.mx), i.sm + j.sm};
    }

    inline void ps(int u, int l, int r, T x) { // apply set.
        s[u]=tr[u].mn=tr[u].mx=x, a[u]=0, tr[u].sm=x*(r-l);
    }

    inline void pa(int u, int l, int r, T x) { // apply add.
        (s[u]!=inf ? s[u] : a[u])+=x;
        tr[u].mn+=x, tr[u].mx+=x, tr[u].sm+=x*(r-l);
    }

    inline void ph(int u, int l, int r) { // push
        int m = l+(r-l)/2, lc = u<<1, rc = lc|1;
        if(s[u]!=inf)
            ps(lc, l, m, s[u]), ps(rc, m, r, s[u]), s[u]=inf;
        if(a[u])
            pa(lc, l, m, a[u]), pa(rc, m, r, a[u]), a[u]=0;
    }
    // set [x, y) to val
    void set(int x, int y, T val, int u, int l, int r) {
        if(x<=l && r<=y) return ps(u, l, r, val);
        ph(u, l, r); int m = l+(r-l)/2;
        if(x < m) set(x, y, val, u<<1, l, m);
        if(y > m) set(x, y, val, u<<1|1, m, r);
        tr[u] = f(tr[u<<1], tr[u<<1|1]);
    }
    // Add val to [x, y)
    void add(int x, int y, T val, int u, int l, int r) {
        if(x<=l && r<=y) return pa(u, l, r, val);
        ph(u, l, r); int m = l+(r-l)/2;
        if(x < m) add(x, y, val, u<<1, l, m);
        if(y > m) add(x, y, val, u<<1|1, m, r);
        tr[u] = f(tr[u<<1], tr[u<<1|1]);
    }

    node qry(int x, int y, int u, int l, int r) {
        if(x<=l && r<=y) return tr[u];
        ph(u, l, r); int m = l+(r-l)/2;
        // fully in left child, fully in right child, overlap
        if(y <= m) return qry(x, y, u<<1, l, m);
        if(x >= m) return qry(x, y, u<<1|1, m, r);
        return
            f(qry(x, y, u<<1, l, m), qry(x, y, u<<1|1, m, r));
    }
    // First index in [x, y) >= val
    int fst(T val, int x, int y, int u, int l, int r) {
        if(tr[u].mx < val) return -1; // pruned early
        if(l+1 == r) return l;
        ph(u, l, r); int m = l+(r-l)/2;
        if(x < m) {
            int res = fst(val, x, y, u<<1, l, m);
            if(res != -1) return res;
        }
        if(y > m) return fst(val, x, y, u<<1|1, m, r);
        return -1;
    }

    // ---- PUBLIC WRAPPERS ----
    void set(int x, int y, T val){if(x<y) set(x,y,val,1,0,n); }
    void add(int x, int y, T val){if(x<y) add(x,y,val,1,0,n); }
    node qry(int x, int y)
        {return (x<y) ? qry(x,y,1,0,n) : node{inf,-inf,0}; }
    int fst(T val, int x, int y)
        {return (x<y) ? fst(val,x,y,1,0,n) : -1; }
};
```

SegmentTreeIterative.h

Description: Zero-indexed max-tree. Bounds are inclusive to the left and exclusive to the right. Can be changed by modifying T, f and unit.
Time: $\mathcal{O}(\log N)$

```
using T = ll;
struct Seg {
    int n, m=1;
    vector<T> t;
    const T unit = -inf;   // identity element

    Seg(int _n): n(_n) {
        while(m<n) m<<=1;
        t.assign(2*m, unit);
    }
    T f(T a, T b) { return max(a, b); } // operation

    void upd(int p, T v) {
        for(t[p+=m]=v;  p>1;  p>>=1)
            t[p>>1] = f(t[p&~1], t[p|1]);// parent, lef, right
    }
    T qry(int l, int r) {   // [l, r)
        T L=unit, R=unit;
        for(l+=m, r+=m;  l<r;  l>>=1, r>>=1) {
            if(l&1) L = f(L, t[l++]);
            if(r&1) R = f(t[--r], R);
        }
        return f(L,R);
    }
    // first position with value >= v
    int first(T v) {
        if(t[1] < v) return -1;
        int p = 1;
        while(p < m) {
            if(t[2*p] >= v) p = 2*p;
            else p = 2*p+1;
        }
        int res = p - m;
        return (res < n) ? res : -1;
    }
};
```

UnionFind.h

```
struct DSU {
    vector<int> p, _sz;
    DSU(int n) {
        p.resize(n); _sz.assign(n, 1);
        iota(p.begin(), p.end(), 0);
    }
    int find(int i) {
        if (p[i] == i) return i;
        return p[i] = find(p[i]); // Path compress
    }
    bool join(int i, int j) {
        int pi = find(i); int pj = find(j);
        if (pi == pj) return false;
        if (_sz[pi] < _sz[pj]) swap(pi, pj);
        p[pj] = pi;  _sz[pi] += _sz[pj];
        return true;
    }
};
```

SegmentTreeRecursive.h

```
struct SegTree { // 0 based
    int n;
    vector<int> t;
    SegTree(int n) : n(n), t(4 * n, 0) {}
    void u(int node, int l, int r, int i, int v) {
        if (l == r) { t[node] = v; return; }
        int m = (l + r) / 2;
        if (i <= m) u(2 * node, l, m, i, v);
        else u(2 * node + 1, m + 1, r, i, v);
        t[node] = max(t[2 * node], t[2 * node + 1]);
    }
    int q(int node, int l, int r, int ql, int qr) {
        if (ql <= l && r <= qr) return t[node]; // Fully in
        if (r < ql || l > qr) return -inf;      // Outside
        int m = (l + r) / 2;
        return max(q(2 * node, l, m, ql, qr), q(2 * node + 1, m
              + 1, r, ql, qr));
    }
    int f(int node, int l, int r, int ql, int qr, int x) {
        if (l > qr || r < ql || t[node] < x) return -1;
        if (l == r) return l;       // Target leaf index found
        int m = (l + r) / 2;
        int res = f(2 * node, l, m, ql, qr, x); // left first
        if (res != -1) return res;
        return f(2 * node + 1, m + 1, r, ql, qr, x); // right
    }
    // Update array[i] = v
    void u(int i, int v) { u(1, 0, n - 1, i, v); }
    // Query max value in range [ql, qr]
    int q(int ql, int qr) { return q(1, 0, n - 1, ql, qr); }
    // Find first index in range [ql, qr] where value >= x (-1
         if none)
    int f(int ql, int qr, int x) { return f(1, 0, n - 1, ql, qr
         , x); }
};
int main() {
    vector<int> a = {1, 3, 2, 5, 4, 7};
    SegTree st(a.size());
    for(int i = 0; i < a.size(); i++) st.u(i, a[i]);
    cout << st.q(1, 4) << "\n"; // Max in [1, 4] -> 5
    cout << st.f(0, 5, 4) << "\n";// 1st >= 4 in [0, 5] : idx 3
    return 0;
}
```

UnionFindRollback.h

Description: Disjoint-set data structure with undo. If undo is not needed, skip : st, time() and rollback().
Usage: `int t = uf.time(); ...; uf.rollback(t);`
Time: $\mathcal{O}(\log(N))$

```
// c = current no. of components
// time() returns current checkpoint
// rollback(x) rolls back to checkpoint x
struct DSU {
  vi e; vector<pii> st; int c;
  DSU(int n) : e(n, -1), c(n) {}
  int size(int x) { return -e[find(x)]; }
  int find(int x) { return e[x] < 0 ? x : find(e[x]); }
  int time() { return sz(st); }
  void rollback(int t) {
    c += (time() - t) / 2;
    for (int i = time(); i --> t; )
      e[st[i].first] = st[i].second;
    st.resize(t);
  }
  bool join(int a, int b) {
    a = find(a), b = find(b);
    if (a == b) return false;
    if (e[a] > e[b]) swap(a, b);
    st.push_back({a, e[a]});
    st.push_back({b, e[b]});
    e[a] += e[b]; e[b] = a;
    --c;
    return true;
  }
};
```

FenwickTree.h

Description: Computes partial sums a[0] + a[1] + ... + a[pos - 1], and updates single elements a[i], taking the difference between the old and new value.
Time: Both operations are $\mathcal{O}(\log N)$.

```
/*
  For range add + point qry:
  to add x to [l,r] do: update(l,x) and update(r+1,-x)
  then point qry at pos becomes : query(pos+1)
*/
struct FT {
  vector<ll> s;
  FT(int n) : s(n) {}
  void update(int pos, ll dif) { // a[pos] += dif
    for (; pos < sz(s); pos |= pos + 1) s[pos] += dif;
  }
  ll query(int pos) { // sum of values in [0, pos)
    ll res = 0;
    for (; pos > 0; pos &= pos - 1) res += s[pos-1];
    return res;
  }
  int lower_bound(ll sum) {// min pos st sum of [0, pos] >= sum
    // Returns n if no sum is >= sum, or -1 if empty sum is.
    if (sum <= 0) return -1;
    int pos = 0;
    for (int pw = 1 << 25; pw; pw >>= 1) {
      if (pos + pw <= sz(s) && s[pos + pw-1] < sum)
        pos += pw, sum -= s[pos-1];
    }
    return pos;
  }
};
```

FenwickTree2d.h

Description: Computes sums a[i,j] for all i<I, j<J, and increases single elements a[i,j]. Requires that the elements to be updated are known in advance (call fakeUpdate() before init()).
Time: $\mathcal{O}\left(\log^2 N\right)$. (Use persistent segment trees for $\mathcal{O}(\log N)$.)

"FenwickTree.h"

```
struct FT2 {
  vector<vi> ys; vector<FT> ft;
  FT2(int limx) : ys(limx) {}
  void fakeUpdate(int x, int y) {
    for (; x < sz(ys); x |= x + 1) ys[x].push_back(y);
  }
  void init() {
    for (vi& v : ys) {
      sort(all(v));
      v.erase(unique(all(v)), v.end());
      ft.emplace_back(sz(v));
    }
  }
  int ind(int x, int y) {
    return (int)(lower_bound(all(ys[x]), y) - ys[x].begin()); }
  void update(int x, int y, ll dif) {
    for (; x < sz(ys); x |= x + 1)
      ft[x].update(ind(x, y), dif);
  }
  ll query(int x, int y) {
    ll sum = 0;
    for (; x; x &= x - 1)
      sum += ft[x-1].query(ind(x-1, y));
    return sum;
  }
};
```

MergeSortTree.h

```
struct MergeSortTree {
    int n;
    vector<vector<int>> tree;
    MergeSortTree(const vector<int>& a) : n(a.size()), tree(4 *
        a.size()) {
        build(a, 1, 0, n - 1);
    }
    void build(const vector<int>& a, int node, int l, int r) {
        if (l == r) {
            tree[node] = {a[l]};
            return;
        }
        int mid = l + (r - l) / 2;
        build(a, 2 * node, l, mid);
        build(a, 2 * node + 1, mid + 1, r);
        merge(tree[2 * node].begin(), tree[2 * node].end(),
              tree[2 * node + 1].begin(), tree[2 * node +
                  1].end(),
              back_inserter(tree[node]));
    }

    // Example Method: Count elements strictly greater than k
        in range [ql, qr]
    int query_gt(int node, int l, int r, int ql, int qr, int k)
        {
        if (qr < l || ql > r) return 0;
        if (ql <= l && r <= qr) {
            auto it = upper_bound(tree[node].begin(), tree[node
                ].end(), k);
            return tree[node].end() - it;
        }
        int mid = l + (r - l) / 2;
        return query_gt(2 * node, l, mid, ql, qr, k) +
               query_gt(2 * node + 1, mid + 1, r, ql, qr, k);
    }
    int query_gt(int ql, int qr, int k) {
        return query_gt(1, 0, n - 1, ql, qr, k);
    }
};
```

LazySegmentTreeImplicit.h

Description: can use set and add together. ranges are all [), EVEN IN CONSTRUCTOR ***
Usage: `Lseg seg(0, n+1); Lseg seg(-50000, 50001)`
Time: $\mathcal{O}(\log N)$.

```
using T = ll;
struct Lseg {
    struct node {
        T mn=0, mx=0, sm=0; // remove/add variables as needed
    };
    Lseg *lc=0, *rc=0; node v; int l, r;

    //custom
    // s:set tag (inf=none), a:add tag, v:stored node values
    T s=inf, a=0;

    node f(node i, node j) { // merge function  // custom
        return {min(i.mn, j.mn), max(i.mx, j.mx), i.sm + j.sm};
    }

    // Huge interval of neutrals
    Lseg(int _l, int _r): l(_l), r(_r) {}

    void ps(T x) { // apply set  // custom
        s=v.mn=v.mx=x, a=0, v.sm=x*(r-l);
    }

    void pa(T x) { // apply add  //custom
    (s!=inf ? s : a)+=x, v.mn+=x, v.mx+=x, v.sm+=x*(r-l);
  }

    void ph() { // push
        int m = l+(r-l)/2;
        if(!lc) lc=new Lseg(l, m), rc=new Lseg(m, r);
        if(s!=inf) lc->ps(s), rc->ps(s), s=inf;  //custom
        if(a) lc->pa(a), rc->pa(a), a=0; //custom
    }

    void set(int x, int y, T val) { // set [x, y) to val
        if(y<=l || r<=x) return;
        if(x<=l && r<=y) ps(val); //custom
        else ph(), lc->set(x, y, val), rc->set(x, y, val), \
        v=f(lc->v, rc->v);
    }

    void add(int x, int y, T val) { // Add val to [x, y)
        if(y<=l || r<=x) return;
        if(x<=l && r<=y) pa(val); //custom
        else ph(), lc->add(x, y, val), rc->add(x, y, val), \
        v=f(lc->v, rc->v);
    }

    node qry(int x, int y) {
        if(y<=l || r<=x) return {inf,-inf,0};
        if(x<=l && r<=y) return v;
        ph(); return f(lc->qry(x, y), rc->qry(x, y));
    }

    // First index in [x, y) with value >= val
    int fst(T val, int x, int y) {
        if(y<=l || r<=x || v.mx<val) return -1;
        if(l+1 == r) return l;
        ph(); int res = lc->fst(val, x, y);
```

```
        return (res != -1) ? res : rc->fst(val, x, y);
    }
    ~Lseg() { delete lc; delete rc; }
};
```

Matrix.h

```
template<class T> struct Matrix {
    typedef Matrix M;
    int n;
    vector<vector<T>> d;
    Matrix(int _n = 0) : n(_n), d(_n, vector<T>(_n, 0)) {}
    static M identity(int n) {
        M I(n);
        rep(i,0,n) I.d[i][i] = 1;
        return I;
    }
    M operator*(const M& m) const {
        M a(n);
        rep(i,0,n) rep(j,0,n)
            rep(k,0,n) {
                a.d[i][k] = (a.d[i][k] + d[i][j] * m.d[j][k]) %
                      mod;
            }
        return a;
    }
    vector<T> operator*(const vector<T>& vec) const {
        vector<T> ret(n, 0);
        rep(i,0,n) rep(j,0,n) {
            ret[i] = (ret[i] + d[i][j] * vec[j]) % mod;
        }
        return ret;
    }
    M operator^(ll p) const {
        assert(p >= 0);
        M a = identity(n), b(*this);
        while (p) {
            if (p & 1) a = a * b;
            b = b * b;
            p >>= 1;
        }
        return a;
    }
};
```

LineContainer.h

Description: Container where you can add lines of the form kx+m, and query maximum values at points x. Useful for dynamic programming ("convex hull trick").
Time: $\mathcal{O}(\log N)$

```
struct Line {
  mutable ll k, m, p;
  bool operator<(const Line& o) const { return k < o.k; }
  bool operator<(ll x) const { return p < x; }
};
struct LineContainer : multiset<Line, less<>> {
  // (for doubles, use inf = 1/.0, div(a,b) = a/b)
  static const ll inf = LLONG_MAX;
  ll div(ll a, ll b) { // floored division
    return a / b - ((a ^ b) < 0 && a % b); }
  bool isect(iterator x, iterator y) {
    if (y == end()) return x->p = inf, 0;
    if (x->k == y->k) x->p = x->m > y->m ? inf : -inf;
    else x->p = div(y->m - x->m, x->k - y->k);
    return x->p >= y->p;
  }
  void add(ll k, ll m) {
    auto z = insert({k, m, 0}), y = z++, x = y;
    while (isect(y, z)) z = erase(z);
    if (x != begin() && isect(--x, y)) isect(x, y = erase(y));
    while ((y = x) != begin() && (--x)->p >= y->p)
      isect(x, erase(y));
  }
  ll query(ll x) {
    assert(!empty());
    auto l = *lower_bound(x);
    return l.k * x + l.m;
  }
};

using lt = __int128;
struct Line { ll k, m; };
struct CHT {
    vector<Line> h; int p = 0;
    // Returns true if l2 becomes redundant between l1 and l3
    bool bad(Line l1, Line l2, Line l3) {
        return (lt)(l2.m - l1.m) * (l1.k - l3.k) \
     >= (lt)(l3.m - l1.m) * (l1.k - l2.k);
    }
    // Slopes (k) must be added in non-decreasing order
    void add(ll k, ll m) {
        if (!h.empty() && h.back().k == k) {
            if (h.back().m >= m) return;
            h.pop_back();
        }
        while(h.size()>=2 && bad(h[h.size()-2],h.back(),{k,m}))
            h.pop_back();
        h.push_back({k, m});
    }
    // O(1) query: requires x to be non-decreasing
    ll query(ll x) {
        p = min(p, (int)h.size() - 1);
        while (p < h.size() - 1 && \
     h[p].k * x + h[p].m <= h[p+1].k * x + h[p+1].m)
            p++;
        return h[p].k * x + h[p].m;
    }
    // O(log N) query: if x is random/not monotonic
    ll query_bs(ll x) {
        int l = 0, r = h.size() - 1;
        while (l < r) {
            int mid = (l + r) / 2;
            if (h[mid].k * x + h[mid].m < \
         h[mid+1].k * x + h[mid+1].m) l = mid + 1;
            else r = mid;
        }
        return h[l].k * x + h[l].m;
    }
};
```

Treap.h

```
// Indexing is 0-based
struct Treap {
  struct Node {
    int val, sum, mn, mx, size, prior, lazy, set;
    bool has_set = false, rev = false;
    Node *l = nullptr, *r = nullptr;
    Node(int v):val(v),sum(v),mn(v),mx(v),size(1), prior(rand()
        ), lazy(0), set(0){}
  };

  Node* root = nullptr;

  int _sz(Node* t) { return t ? t->size : 0; }
  int sum(Node* t) { return t ? t->sum : 0; }
  int mn(Node* t) { return t ? t->mn : INT_MAX; }
  int mx(Node* t) { return t ? t->mx : INT_MIN; }

  void push(Node* t) {
    if (!t) return;
    if (t->has_set) {
      t->val = t->set;
      t->sum = t->set * _sz(t);
      t->mn = t->mx = t->set;
      if (t->l) t->l->set = t->set, t->l->has_set = 1, t->l->
          lazy = 0;
      if (t->r) t->r->set = t->set, t->r->has_set = 1, t->r->
          lazy = 0;
      t->has_set = 0;
    }
    if (t->lazy) {
      t->val += t->lazy;
      t->sum += t->lazy * _sz(t);
      t->mn += t->lazy; t->mx += t->lazy;
      if (t->l) t->l->lazy += t->lazy;
      if (t->r) t->r->lazy += t->lazy;
      t->lazy = 0;
    }
    if (t->rev) {
      swap(t->l, t->r);
      if (t->l) t->l->rev ^= 1;
      if (t->r) t->r->rev ^= 1;
      t->rev = 0;
    }
  }

  void upd(Node* t) {
    if (!t) return;
    push(t->l); push(t->r);
    t->size = 1 + _sz(t->l) + _sz(t->r);
    t->sum = t->val + sum(t->l) + sum(t->r);
    t->mn = min(t->val, min(mn(t->l), mn(t->r)));
    t->mx = max(t->val, max(mx(t->l), mx(t->r)));
  }

  void split(Node* t, int k, Node*& l, Node*& r) {
    // l holds first k indices [0...k-1]
    // r holds rem. indices [k...n-1]

    if (!t) { l = r = nullptr; return; }
    push(t);
    if (_sz(t->l) < k) {
      split(t->r, k - _sz(t->l) - 1, t->r, r);
      l = t;
    } else {
      split(t->l, k, l, t->l);
      r = t;
    }
    upd(t);
  }

  Node* merge(Node* l, Node* r) {
    push(l); push(r);
    if (!l || !r) return l ? l : r;
    if (l->prior > r->prior) {
      l->r = merge(l->r, r); upd(l); return l;
    } else {
      r->l = merge(l, r->l); upd(r); return r;
    }
  }

// inserts val at index pos [0 based], shifting the rest right
  void insert(int pos, int val) {
    Node *t1, *t2;
    split(root, pos, t1, t2);
```

```
  root = merge(merge(t1, new Node(val)), t2);
}

// Range add [l, r]
void range_add(int l, int r, int v) {
  Node *t1, *t2, *t3; // [0...l-1], [l...r], [r+1...n-1]
  split(root, r+1, t2, t3);
  split(t2, l, t1, t2);
  if (t2) t2->lazy += v;
  root = merge(merge(t1, t2), t3);
}

// Range sum [l, r]
int range_sum(int l, int r) {
  Node *t1, *t2, *t3;
  split(root, r+1, t2, t3);
  split(t2, l, t1, t2);
  int ans = sum(t2);
  root = merge(merge(t1, t2), t3);
  return ans;
}

void range_set(int l, int r, int v) {
  Node *t1, *t2, *t3;
  split(root, r + 1, t2, t3); split(t2, l, t1, t2);
  if (t2) t2->set = v, t2->has_set = 1, t2->lazy = 0;
  root = merge(merge(t1, t2), t3);
}

int range_min(int l, int r) {
  Node *t1, *t2, *t3;
  split(root, r + 1, t2, t3); split(t2, l, t1, t2);
  int ans = mn(t2);
  root = merge(merge(t1, t2), t3);
  return ans;
}

int range_max(int l, int r) {
  Node *t1, *t2, *t3;
  split(root, r + 1, t2, t3); split(t2, l, t1, t2);
  int ans = mx(t2);
  root = merge(merge(t1, t2), t3);
  return ans;
}

int at(int pos) {
  Node* t = root;
  while (t) {
    push(t);
    if (_sz(t->l) == pos) return t->val;
    if (pos < _sz(t->l)) t = t->l;
    else pos -= _sz(t->l) + 1, t = t->r;
  }
  return 0;
}

int kth(int k) { return at(k); }

void range_reverse(int l, int r) {
  Node *t1, *t2, *t3;
  split(root, r + 1, t2, t3); split(t2, l, t1, t2);
  if (t2) t2->rev ^= 1;
  root = merge(merge(t1, t2), t3);
}

void erase(int pos) {
// [0...pos-1], [pos...pos], [pos+1...n-1]
  Node *t1, *t2, *t3;
  split(root, pos + 1, t2, t3);
  split(t2, pos, t1, t2);
  delete t2;
  root = merge(t1, t3);
}

// add functions as needed :

void f(ll a, ll b) {
  if(a >= b) return;

// both cases can be generalized to ...|block|...|block|...
// any of the ... can be null, no problem

  Node *t1, *t2, *t3, *t4, *t5;
  int blocksize = min(n-b, b-a);
  split(root, b+blocksize, t4, t5);
  split(t4, b, t3, t4);
  split(t3, a+blocksize, t2, t3);
  split(t2, a, t1, t2);
  Node *res = merge(t1, t4);
  res = merge(res, t3);
  res = merge(res, t2);
  res = merge(res, t5);
  root = res;
}

void print() {
  print(root); cout << nl;
}
void print(Node* root) { // simply in-order
  if(!root) return;
  print(root->l);
  cout << root->val << gp;
  print(root->r);
}
};
```

RMQ.h

Description: Range Minimum Queries on an array. Returns min(V[a], V[a + 1], ... V[b - 1]) in constant time.
Usage: `RMQ rmq(values);`
`rmq.query(inclusive, exclusive);`
Time: $\mathcal{O}(|V| \log |V| + Q)$

```
template<class T>
struct RMQ {
    const vector<T>& V;
    vector<vector<int>> jmp;
    RMQ(const vector<T>& V) : V(V), jmp(1, vector<int>(sz(V)))
        {
        rep(i,0,sz(V)) jmp[0][i] = i;
        for (int pw = 1, k = 1; pw * 2 <= sz(V); pw *= 2, ++k)
            {
            jmp.emplace_back(sz(V) - pw * 2 + 1);
            rep(j,0,sz(jmp[k])) {
                int a = jmp[k - 1][j];
                int b = jmp[k - 1][j + pw];
                jmp[k][j] = (V[a] <= V[b] ? a : b);
            }
        }
    }
    int queryIdx(int a, int b) { // leftmost for tie breaking
        assert(a < b);
        int dep = 31 - __builtin_clz(b - a);
        int i = jmp[dep][a];
        int j = jmp[dep][b - (1 << dep)];
        return (V[i] <= V[j] ? i : j); // use < for rightmost
    }
    T query(int a, int b) {
        return V[queryIdx(a, b)];
    }
};
```

MoAlgorithm.h

Description: Answer interval or tree path queries by finding an approximate TSP through the queries, and moving from one query to the next by adding/removing points at the ends. If values are on tree edges, change `step` to add/remove the edge (a, c) and remove the initial `add` call (but keep `in`).
Time: $\mathcal{O}\left(N\sqrt{Q}\right)$

```
void add(int ind, int end) { ... } // add a[ind] (end = 0 or 1)
void del(int ind, int end) { ... } // remove a[ind]
int calc() { ... } // compute current answer

vi mo(vector<pii> Q) {
  int L = 0, R = 0, blk = 350; // ~N/sqrt(Q)
  vi s(sz(Q)), res = s;
#define K(x) pii(x.first/blk, x.second ^ -(x.first/blk & 1))
  iota(all(s), 0);
  sort(all(s), [&](int s, int t){ return K(Q[s]) < K(Q[t]); });
  for (int qi : s) {
    pii q = Q[qi];
    while (L > q.first) add(--L, 0);
    while (R < q.second) add(R++, 1);
    while (L < q.first) del(L++, 0);
    while (R > q.second) del(--R, 1);
    res[qi] = calc();
  }
  return res;
}

vi moTree(vector<array<int, 2>> Q, vector<vi>& ed, int root=0){
  int N = sz(ed), pos[2] = {}, blk = 350; // ~N/sqrt(Q)
  vi s(sz(Q)), res = s, I(N), L(N), R(N), in(N), par(N);
  add(0, 0), in[0] = 1;
  auto dfs = [&](int x, int p, int dep, auto& f) -> void {
    par[x] = p;
    L[x] = N;
    if (dep) I[x] = N++;
    for (int y : ed[x]) if (y != p) f(y, x, !dep, f);
    if (!dep) I[x] = N++;
    R[x] = N;
  };
  dfs(root, -1, 0, dfs);
#define K(x) pii(I[x[0]] / blk, I[x[1]] ^ -(I[x[0]] / blk & 1))
  iota(all(s), 0);
  sort(all(s), [&](int s, int t){ return K(Q[s]) < K(Q[t]); });
  for (int qi : s) rep(end,0,2) {
    int &a = pos[end], b = Q[qi][end], i = 0;
#define step(c) { if (in[c]) { del(a, end); in[a] = 0; } \
                  else { add(c, end); in[c] = 1; } a = c; }
    while (!(L[b] <= L[a] && R[a] <= R[b]))
      I[i++] = b, b = par[b];
    while (a != b) step(par[a]);
    while (i--) step(I[i]);
    if (end) res[qi] = calc();
  }
  return res;
}
```

Dynamic Programming (2)

LIS.h

Description: Compute indices for the longest increasing subsequence.
Time: $\mathcal{O}(N \log N)$

```
template<class I> vi lis(const vector<I>& S) {
```

```
  if (S.empty()) return {};
  vi prev(sz(S));
  typedef pair<I, int> p;
  vector<p> res;
  rep(i,0,sz(S)) {
    // change 0 -> i for longest non-decreasing subsequence
    auto it = lower_bound(all(res), p{S[i], 0});
    if (it == res.end()) res.emplace_back(), it = res.end()-1;
    *it = {S[i], i};
    prev[i] = it == res.begin() ? 0 : (it-1)->second;
  }
  int L = sz(res), cur = res.back().second;
  vi ans(L);
  while (L--) ans[L] = cur, cur = prev[cur];
  return ans;
}
```

DigitDP.h

```
// count tuples (a1, a2, b1, b2) where :
// a2 < a1 <= a, b2 < b1 <= b, and a1^a2 = b1^b2
// [pos][a1_eq_a][b1_eq_b][a2_eq_a1][b2_eq_b1]
mint dp[30][2][2][2][2];

// base case : take one bit higher than max possible msb.
// all bits must be zero here. all tight. only one base case :)
dp[29][1][1][1][1] = 1;

R(pos, 29, 1) {
  int nexta = isSet(a, pos - 1), nextb = isSet(b, pos - 1);
  L(a1_eq_a,0,1) L(b1_eq_b,0,1) L(a2_eq_a1,0,1) L(b2_eq_b1,0,1)
      {
    L(nexta1,0,1) L(nextb1,0,1) L(nexta2,0,1) L(nextb2,0,1){
      if (nexta1 ^ nextb1 ^ nexta2 ^ nextb2) continue;

      if (a1_eq_a && nexta == 0 && nexta1) continue;
      if (b1_eq_b && nextb == 0 && nextb1) continue;
      if (a2_eq_a1 && nexta1 == 0 && nexta2) continue;
      if (b2_eq_b1 && nextb1 == 0 && nextb2) continue;

      int na1_eq_a = a1_eq_a && (nexta == nexta1);
      int nb1_eq_b = b1_eq_b && (nextb == nextb1);
      int na2_eq_a1 = a2_eq_a1 && (nexta2 == nexta1);
      int nb2_eq_b1 = b2_eq_b1 && (nextb2 == nextb1);

      dp[pos - 1][na1_eq_a][nb1_eq_b][na2_eq_a1][nb2_eq_b1] +=
          dp[pos][a1_eq_a][b1_eq_b][a2_eq_a1][b2_eq_b1];
    }
  }
}

mint valid_tuples;
L(a1_eq_a, 0, 1) L(b1_eq_b, 0, 1) {
  valid_tuples += dp[0][a1_eq_a][b1_eq_b][0][0];
}
```

SOSDP.h

```
//  for problems like sum/count involving submaks/supermask,
//   we use sos dp to avoid overcounting.
//  x|y = x  :  y is submask of x
//  x&y = x  :  y is supermask of x
//  x&y = 0  :  y is submask of ~x

// k = koyta bit max

void fwd1(vll& dp, int k){ // dp[x] = cnt of submask of x
 int N = 1ll<<k;
 L(bit,0,k-1){
  L(mask,0,N-1){
   if(isSet(mask,bit)) dp[mask]+=dp[resetbit(mask,bit)]; } } }

void bak1(vll& dp, int k){ // return from submask count to mask
     count
 int N = 1ll<<k;
 R(bit,19,0){
  R(mask,N-1,0){
   if(isSet(mask,bit)) dp[mask]-=dp[resetbit(mask,bit)]; } } }

void fwd2(vll& dp, int k){ // dp[x] = cnt of supermask of x
 int N = 1ll<<k;
 L(bit,0,19){
  R(mask,N-1,0){
   if(isSet(mask,bit)) dp[resetbit(mask,bit)]+=dp[mask]; } } }

void bak2(vll& dp, int k){ // return from supermask count to
     mask count
 int N = 1ll<<k;
 R(bit,19,0){
  L(mask,0,N-1){
   if(isSet(mask,bit)) dp[resetbit(mask,bit)]-=dp[mask]; } } }
```

SubsetSumSqrt.h

```
main:
// Sum of elements <= N implies that every element is <= N
vector<int> freq(N + 1, 0);
for (int i = 0; i < N; i++) {
    int x;  cin >> x;  freq[x]++;
}
vector<pair<int, int>> compressed;
for (int i = 1; i <= N; i++) {
    if (freq[i] > 0) compressed.emplace_back(i, freq[i]);
}
vector<int> dp(N + 1, 0);  dp[0] = 1;
for (const auto &[w, k] : compressed) {
    vector<int> ndp = dp;
    for (int p = 0; p < w; p++) {
        int sum = 0;
        for(int mult = p, cnt = 0; mult<=N; mult += w, cnt++){
            if (cnt > k) {
                sum -= dp[mult - w * cnt];  cnt--;
            }
            if (sum > 0) ndp[mult] = 1;  sum += dp[mult];
        }
    }
    swap(dp, ndp);
}
cout << "Possible subset sums are:\n";
for (int i = 0; i <= N; i++) {
    if (dp[i] > 0) cout << i << " ";
}
```

DivideAndConquerDP.h

Description: the dnc sol for cses subarray squares
Time: k*n*log(n) here

```
/*
  dnc dp can be applid for
    dp[i] = min( dp[j] + cost(j+1..i) ) for all j<i
  if opt(i) <= opt(i-1) is satisfied
    where opt(i) = smallest j among all the best j for i
  this may or may not be obvious
    [if you cant prove, try bruteforcing too see if it holds]
  one (not only) case when this is true is:
    cost function satisfies quadrangle inequality:
    cost(a..c) + cost(b..d) <= cost(a..d) + cost(b..c)
    where a <= b <= c <= d
  the cost function (pref[i]-pref[j])^2 satisfies this
  this runs in nlogn for each k. so k*nlogn total
*/
const int MXN = 3005, inf = 2e18;
ll pref[MXN], dp[MXN][MXN];//dp[i][k]=divide upto i int k segs
ll cost(int l, int r) { // change
    int sum = pref[r] - pref[l-1]; return sum*sum;
}
void dnc(int k, int l, int r, int optl, int optr) {
    if (l > r) return;
    int mid = (l + r) / 2;
    pair<int, int> best = {inf, l};
    for (int j = optl; j <= min(mid - 1, optr); j++) {
        best = min(best, make_pair(dp[j][k-1] + cost(j+1, mid),
              j));
    }
    dp[mid][k] = best.first;
    int opt = best.second;
    dnc(k, l, mid - 1, optl, opt);
    dnc(k, mid + 1, r, opt, optr);
}
usage:
dp[0][0] = 0;
for (int i = 1; i <= n; i++) dp[i][0] = inf;
for (int k = 1; k <= _K; k++) dnc(k, 1, n, 0, n - 1);
// [optl, optr] = possible values of j. starts with [0,n-1]
```

KnuthDP.h

Description: When doing DP on intervals: $a[i][j] = \min_{i<k<j}(a[i][k] + a[k][j]) + f(i,j)$, where the (minimal) optimal k increases with both i and j, one can solve intervals in increasing order of length, and search $k = p[i][j]$ for $a[i][j]$ only between $p[i][j-1]$ and $p[i+1][j]$. This is known as Knuth DP. Sufficient criteria for this are if $f(b,c) \le f(a,d)$ and $f(a,c) + f(b,d) \le f(a,d) + f(b,c)$ for all $a \le b \le c \le d$. Consider also: LineContainer (ch. Data structures), monotone queues, ternary search.
Time: $\mathcal{O}\left(N^2\right)$

BinaryGrouping.h

Description: Split bounded-multiplicity items into 0-1 bundles. Each item is given as {weight, value, count}; the result contains pairs of total weight and value suitable for 0-1 knapsack.
Time: Bundle generation $\mathcal{O}\left(\sum_i \log k_i\right)$; using the bundles in 0-1 knapsack takes $\mathcal{O}\left(W \sum_i \log k_i\right)$.

```
vector<pair<int, int>> binaryGrouping(vector<array<int, 3>>
     items) {
  vector<pair<int, int>> res;
  for (auto [w, v, k] : items) {
    if (!k) continue;
    for (int c = 1; k > c; c *= 2) {
      k -= c;
      res.emplace_back(c * w, c * v);
    }
    res.emplace_back(k * w, k * v);
  }
  return res;
}
```

Number theory (3)

3.1 Modular arithmetic

ModPow.h

```
const ll mod = 1000000007; // faster if const

ll modpow(ll b, ll e) {
```

```
  ll ans = 1;
  for (; e; b = b * b % mod, e /= 2)
    if (e & 1) ans = ans * b % mod;
  return ans;
}
```

ModLog.h

Description: Returns the smallest $x > 0$ s.t. $a^x = b \pmod m$, or -1 if no such x exists. modLog(a,1,m) can be used to calculate the order of a.
Time: $\mathcal{O}\left(\sqrt{m}\right)$

```
ll modLog(ll a, ll b, ll m) {
  ll n = (ll) sqrt(m) + 1, e = 1, f = 1, j = 1;
  unordered_map<ll, ll> A;
  while (j <= n && (e = f = e * a % m) != b % m)
    A[e * b % m] = j++;
  if (e == b % m) return j;
  if (__gcd(m, e) == __gcd(m, b))
    rep(i,2,n+2) if (A.count(e = e * f % m))
      return n * i - A[e];
  return -1;
}
```

ModSum.h

Description: Sums of mod'ed arithmetic progressions.
modsum(to, c, k, m) $= \sum_{i=0}^{to-1} (ki + c)\%m$. divsum is similar but for floored division.
Time: $\log(m)$, with a large constant.

```
typedef unsigned long long ull;
ull sumsq(ull to) { return to / 2 * ((to-1) | 1); }
ull divsum(ull to, ull c, ull k, ull m) {
  ull res = k / m * sumsq(to) + c / m * to;
  k %= m; c %= m;
  if (!k) return res;
  ull to2 = (to * k + c) / m;
  return res + (to - 1) * to2 - divsum(to2, m-1 - c, m, k);
}
ll modsum(ull to, ll c, ll k, ll m) {
  c = ((c % m) + m) % m;
  k = ((k % m) + m) % m;
  return to * c + k * sumsq(to) - m * divsum(to, c, k, m);
}
```

ModMulLL.h

Description: Calculate $a \cdot b \bmod c$ (or $a^b \bmod c$) for $0 \le a, b \le c \le 7.2 \cdot 10^{18}$.
Time: $\mathcal{O}(1)$ for modmul, $\mathcal{O}(\log b)$ for modpow

```
typedef unsigned long long ull;
ull modmul(ull a, ull b, ull M) {
  ll ret = a * b - M * ull(1.L / M * a * b);
  return ret + M * (ret < 0) - M * (ret >= (ll)M);
}
ull modpow(ull b, ull e, ull mod) {
  ull ans = 1;
  for (; e; b = modmul(b, b, mod), e /= 2)
    if (e & 1) ans = modmul(ans, b, mod);
  return ans;
}
```

ModSqrt.h

Description: Tonelli-Shanks algorithm for modular square roots. Finds x s.t. $x^2 = a \pmod p$ ($-x$ gives the other solution).
Time: $\mathcal{O}\left(\log^2 p\right)$ worst case, $\mathcal{O}(\log p)$ for most p

"ModPow.h"

```
ll sqrt(ll a, ll p) {
  a %= p; if (a < 0) a += p;
  if (a == 0) return 0;
  assert(modpow(a, (p-1)/2, p) == 1); // else no solution
  if (p % 4 == 3) return modpow(a, (p+1)/4, p);
  // a^(n+3)/8 or 2^(n+3)/8 * 2^(n-1)/4 works if p % 8 == 5
  ll s = p - 1, n = 2;
  int r = 0, m;
  while (s % 2 == 0)
    ++r, s /= 2;
  while (modpow(n, (p - 1) / 2, p) != p - 1) ++n;
  ll x = modpow(a, (s + 1) / 2, p);
  ll b = modpow(a, s, p), g = modpow(n, s, p);
  for (;; r = m) {
    ll t = b;
    for (m = 0; m < r && t != 1; ++m)
      t = t * t % p;
    if (m == 0) return x;
    ll gs = modpow(g, 1LL << (r - m - 1), p);
    g = gs * gs % p;
    x = x * gs % p;
    b = b * g % p;
  }
}
```

3.2 Primality

FastEratosthenes.h

Description: Prime sieve for generating all primes smaller than LIM.
Time: LIM=1e9 $\approx$ 1.5s

```
const int LIM = 1e6;
bitset<LIM> isPrime;
vi eratosthenes() {
  const int S = (int)round(sqrt(LIM)), R = LIM / 2;
  vi pr = {2}, sieve(S+1); pr.reserve(int(LIM/log(LIM)*1.1));
  vector<pii> cp;
  for (int i = 3; i <= S; i += 2) if (!sieve[i]) {
    cp.push_back({i, i * i / 2});
    for (int j = i * i; j <= S; j += 2 * i) sieve[j] = 1;
  }
  for (int L = 1; L <= R; L += S) {
    array<bool, S> block{};
    for (auto &[p, idx] : cp)
      for (int i=idx; i < S+L; idx = (i+=p)) block[i-L] = 1;
    rep(i,0,min(S, R - L))
      if (!block[i]) pr.push_back((L + i) * 2 + 1);
  }
  for (int i : pr) isPrime[i] = 1;
  return pr;
}
```

MillerRabin.h

Description: Deterministic Miller-Rabin primality test. Guaranteed to work for numbers up to $7 \cdot 10^{18}$; for larger numbers, use Python and extend A randomly.
Time: 7 times the complexity of $a^b \mod c$.

"ModMulLL.h"

```
bool isPrime(ull n) {
  if (n < 2 || n % 6 % 4 != 1) return (n | 1) == 3;
  ull A[] = {2, 325, 9375, 28178, 450775, 9780504, 1795265022},
      s = __builtin_ctzll(n-1), d = n >> s;
  for (ull a : A) {   // ^ count trailing zeroes
    ull p = modpow(a%n, d, n), i = s;
    while (p != 1 && p != n - 1 && a % n && i--)
      p = modmul(p, p, n);
    if (p != n-1 && i != s) return 0;
  }
  return 1;
}
```

PollardRho.h

Description: Pollard-rho randomized factorization algorithm. Returns prime factors of a number, in arbitrary order (e.g. 2299 -> {11, 19, 11}).
Time: $\mathcal{O}\left(n^{1/4}\right)$, less for numbers with small factors.

"ModMulLL.h", "MillerRabin.h"

```
ull pollard(ull n) {
  ull x = 0, y = 0, t = 30, prd = 2, i = 1, q;
  auto f = [&](ull x) { return modmul(x, x, n) + i; };
  while (t++ % 40 || __gcd(prd, n) == 1) {
    if (x == y) x = ++i, y = f(x);
    if ((q = modmul(prd, max(x,y) - min(x,y), n))) prd = q;
    x = f(x), y = f(f(y));
  }
  return __gcd(prd, n);
}
vector<ull> factor(ull n) {
  if (n == 1) return {};
  if (isPrime(n)) return {n};
  ull x = pollard(n);
  auto l = factor(x), r = factor(n / x);
  l.insert(l.end(), all(r));
  return l;
}
```

spf.h

```
const int N = 1e7;
vector<int> spf(N);
void prep(){ // modified sieve
    L(i, 2, N-1) {
        if(spf[i] == 0) {
            spf[i] = i;
            for(ll j = i * i; j <= N-1; j += i) {
                if(spf[j] == 0) spf[j] = i;    }}}}
// prime factorize [if you need 1e9, use loop upto sqrt]
vector<pll> prime_factorize(ll x) {
    vector<pll> ans;
    while(x!=1) {
        ll cur = spf[x];  ll cnt = 0;
        while(x%cur==0) cnt++, x/=cur;
        ans.push_back({cur, cnt});   }
    return ans;   }
```

LinearSievePhiMuSpf.h

```
const int N = 1000000;
vector<int> primes;
int spf[N + 1];  int phi[N + 1];  int mu[N + 1];
bool is_composite[N + 1];
// int func[MAXN], cnt[MAXN]; // Extension

void linear_sieve(int n) {
    std::fill (is_composite, is_composite + n, false);
    phi[1] = 1;  mu[1] = 1;
    for (int i = 2; i <= n; i++) {
        if (!is_composite[i]) {
            primes.push_back(i);
            spf[i] = i;  phi[i] = i - 1;  mu[i] = -1;
        }
        for (int p : primes) {
            if (i * 1LL * p > n) break;
            is_composite[i * p] = true;
            spf[i * p] = p;

            if (i % p == 0) {
                phi[i * p] = phi[i] * p;
                mu[i * p] = 0;
            // func[i*prime[j]] = func[i]/cnt[i] * (cnt[i]+1);
            // cnt[i * prime[j]] = cnt[i] + 1;
```

```
                break;
            } else {
                phi[i * p] = phi[i] * (p - 1);
                mu[i * p] = -mu[i];
            // func[i * prime[j]] = func[i] * func[prime[j]];
            // cnt[i * prime[j]] = 1;
            }
        }
    }
}
```

3.3 Divisibility

euclid.h

Description: Finds two integers x and y, such that $ax + by = \gcd(a, b)$. If you just need gcd, use the built in __gcd instead. If a and b are coprime, then x is the inverse of $a \pmod b$.

```
// Extended Euclid
ll euclid(ll a, ll b, ll &x, ll &y) {
    if (b == 0) { x = 1; y = 0; return a; }
    ll x1, y1, g = euclid(b, a % b, x1, y1);
    x = y1; y = x1 - (a / b) * y1;
    return g;
}
```

Diophantine.h

```
// Solve ax + by = c
bool diophantine(ll a, ll b, ll c, ll &x, ll &y, ll &g) {
    if (a == 0 && b == 0) {
        if (c == 0) { x = 0; y = 0; g = 0; return true; }
        return false;
    }
    g = euclid(abs(a), abs(b), x, y);
    if (c % g != 0) return false;
    if (a < 0) x = -x;
    if (b < 0) y = -y;
    x *= (c / g);
    y *= (c / g);
    return true;
}
```

DiophNonNeg.h

```
// Non-negative solution (x >= 0, y >= 0)
bool dioph_nonneg(ll a, ll b, ll c, ll &x, ll &y, ll &g) {
    if (!diophantine(a, b, c, x, y, g)) return false;
    if (a == 0) {
        if (b == 0) return c == 0;
        if (y >= 0) { x = 0; return true; }
        return false;
    }
    if (b == 0) {
        if (x >= 0) { y = 0; return true; }
        return false;
    }
    ll dx = b / g;
    ll dy = a / g;
    ll k_min = -INF;
    ll k_max = INF;
    // x + k*dx >= 0  =>  k*dx >= -x
    if (dx > 0) k_min = max(k_min, ceil_div(-x, dx));
    else if (dx < 0) k_max = min(k_max, floor_div(-x, dx));
    else if (-x > 0) return false;
    // y - k*dy >= 0  =>  k*dy <= y
    if (dy > 0) k_max = min(k_max, floor_div(y, dy));
    else if (dy < 0) k_min = max(k_min, ceil_div(y, dy));
    else if (y < 0) return false;
    if (k_min > k_max) return false;
    // Any k in [k_min, k_max] gives a non-negative solution.
    // k_min minimizes x (if dx > 0) or maximizes x (if dx < 0)
    ll k = k_min;
    x += k * dx;
    y -= k * dy;
    return true;
}
```

CRT.h

Description: Chinese Remainder Theorem.
`crt(a, m, b, n)` computes x such that $x \equiv a \pmod m$, $x \equiv b \pmod n$. If $|a| < m$ and $|b| < n$, x will obey $0 \le x < \text{lcm}(m, n)$. Assumes $mn < 2^{62}$.
Time: $\log(n)$

"euclid.h"

```
ll crt(ll a, ll m, ll b, ll n) {
  if (n > m) swap(a, b), swap(m, n);
  ll x, y, g = euclid(m, n, x, y);
  assert((a - b) % g == 0); // else no solution
  x = (b - a) % n * x % n / g * m + a;
  return x < 0 ? x + m*n/g : x;
}
```

3.3.1 Bézout's identity

For $a \neq, b \neq 0$, then $d = gcd(a, b)$ is the smallest positive integer for which there are integer solutions to

$$ax + by = d$$

If (x, y) is one solution, then all solutions are given by

$$\left(x + \frac{kb}{\gcd(a,b)}, y - \frac{ka}{\gcd(a,b)}\right), \quad k \in \mathbb{Z}$$

phiFunction.h

Description: *Euler's* ϕ function is defined as $\phi(n) :=$ # of positive integers $\le n$ that are coprime with n. $\phi(1) = 1$, p prime $\Rightarrow \phi(p^k) = (p-1)p^{k-1}$, m, n coprime $\Rightarrow \phi(mn) = \phi(m)\phi(n)$. If $n = p_1^{k_1} p_2^{k_2} ... p_r^{k_r}$ then $\phi(n) = (p_1 - 1)p_1^{k_1 - 1} ... (p_r - 1)p_r^{k_r - 1}$. $\phi(n) = n \cdot \prod_{p|n}(1 - 1/p)$.
$\sum_{d|n} \phi(d) = n$, $\sum_{1 \le k \le n, \gcd(k,n)=1} k = n\phi(n)/2, n > 1$
Euler's thm: a, n coprime $\Rightarrow a^{\phi(n)} \equiv 1 \pmod n$.
Fermat's little thm: p prime $\Rightarrow a^{p-1} \equiv 1 \pmod p$ $\forall a$.

```
const int LIM = 5000000;
int phi[LIM];

void calculatePhi() {
  rep(i,0,LIM) phi[i] = i&1 ? i : i/2;
  for (int i = 3; i < LIM; i += 2) if(phi[i] == i)
    for (int j = i; j < LIM; j += i) phi[j] -= phi[j] / i;
}
```

3.4 Pythagorean Triples

The Pythagorean triples are uniquely generated by

$$a = k \cdot (m^2 - n^2), \ \ b = k \cdot (2mn), \ \ c = k \cdot (m^2 + n^2),$$

with $m > n > 0$, $k > 0$, $m \perp n$, and either m or n even.

3.5 Primes

$p = 962592769$ is such that $2^{21} \mid p - 1$, which may be useful. For hashing use 970592641 (31-bit number), 31443539979727 (45-bit), 3006703054056749 (52-bit). There are 78498 primes less than 1 000 000.

Primitive roots exist modulo any prime power p^a, except for $p = 2, a > 2$, and there are $\phi(\phi(p^a))$ many. For $p = 2, a > 2$, the group $\mathbb{Z}_{2^a}^\times$ is instead isomorphic to $\mathbb{Z}_2 \times \mathbb{Z}_{2^{a-2}}$.

3.6 Estimates

$\sum_{d|n} d = O(n \log \log n)$.

The number of divisors of n is at most around 100 for $n < 5e4$, 500 for $n < 1e7$, 2000 for $n < 1e10$, 200 000 for $n < 1e19$.

3.7 Mobius Function

$$\mu(n) = \begin{cases} 0 & n \text{ is not square free} \\ 1 & n \text{ has even number of prime factors} \\ -1 & n \text{ has odd number of prime factors} \end{cases}$$

Mobius Inversion:

$$g(n) = \sum_{d|n} f(d) \Leftrightarrow f(n) = \sum_{d|n} \mu(d) g(n/d)$$

Other useful formulas/forms:

$\sum_{d|n} \mu(d) = [n = 1]$ (very useful)

$g(n) = \sum_{n|d} f(d) \Leftrightarrow f(n) = \sum_{n|d} \mu(d/n) g(d)$

$g(n) = \sum_{1 \le m \le n} f(\lfloor \frac{n}{m} \rfloor) \Leftrightarrow f(n) = \sum_{1 \le m \le n} \mu(m) g(\lfloor \frac{n}{m} \rfloor)$

Combinatorial (4)

4.1 Permutations

4.1.1 Factorial

n	1	2	3	4	5	6	7	8	9	10
$n!$	1	2	6	24	120	720	5040	40320	362880	3628800
n	11	12	13	14	15	16	17			
$n!$	4.0e7	4.8e8	6.2e9	8.7e10	1.3e12	2.1e13	3.6e14			
n	20	25	30	40	50	100	150	171		
$n!$	2e18	2e25	3e32	8e47	3e64	9e157	6e262	>DBL_MAX		

IntPerm.h

Description: Permutation -> integer conversion. (Not order preserving.) Integer -> permutation can use a lookup table.
Time: $\mathcal{O}(n)$

```
int permToInt(vi& v) {
  int use = 0, i = 0, r = 0;
  for(int x:v) r = r * ++i + __builtin_popcount(use & -(1<<x)),
    use |= 1 << x;                    // (note: minus, not ~!)
  return r;
}
```

4.1.2 Cycles

Let $g_S(n)$ be the number of n-permutations whose cycle lengths all belong to the set S. Then

$$\sum_{n=0}^{\infty} g_S(n)\frac{x^n}{n!} = \exp\left(\sum_{n\in S}\frac{x^n}{n}\right)$$

4.1.3 Derangements

Permutations of a set such that none of the elements appear in their original position.

$$D(n) = (n-1)(D(n-1)+D(n-2)) = nD(n-1)+(-1)^n = \left\lfloor \frac{n!}{e} \right\rceil$$

4.1.4 Burnside's lemma

Given a group G of symmetries and a set X, the number of elements of X *up to symmetry* equals

$$\frac{1}{|G|}\sum_{g\in G}|X^g|,$$

where X^g are the elements fixed by g ($g.x = x$).

If $f(n)$ counts "configurations" (of some sort) of length n, we can ignore rotational symmetry using $G = \mathbb{Z}_n$ to get

$$g(n) = \frac{1}{n}\sum_{k=0}^{n-1} f(\gcd(n,k)) = \frac{1}{n}\sum_{k|n} f(k)\phi(n/k).$$

4.2 Partitions and subsets

4.2.1 Partition function

Number of ways of writing n as a sum of positive integers, disregarding the order of the summands.

$$p(0) = 1,\ p(n) = \sum_{k\in\mathbb{Z}\setminus\{0\}} (-1)^{k+1} p(n-k(3k-1)/2)$$

$$p(n) \sim 0.145/n \cdot \exp(2.56\sqrt{n})$$

n	0	1	2	3	4	5	6	7	8	9	20	50	100
$p(n)$	1	1	2	3	5	7	11	15	22	30	627	∼2e5	∼2e8

4.2.2 Lucas' Theorem

Let n, m be non-negative integers and p a prime. Write $n = n_k p^k + ... + n_1 p + n_0$ and $m = m_k p^k + ... + m_1 p + m_0$. Then $\binom{n}{m} \equiv \prod_{i=0}^{k}\binom{n_i}{m_i} \pmod p$.

4.2.3 Combinatorial Identities

Hockey Stick Identity:

$$\sum_{i=r}^{n}\binom{i}{r} = \binom{n+1}{r+1}$$

Vandermonde Identity:

$$\binom{m+n}{r} = \sum_{k=0}^{r}\binom{m}{k}\binom{n}{r-k}$$

Others:

$$(x+1)\binom{x}{k-1} = k\binom{x+1}{k}$$

$$\sum_{k=0}^{n}\binom{n}{k}^2 = \binom{2n}{n}$$

$$\sum_{k=0}^{n}\binom{n}{k} = 2^n$$

4.2.4 Binomials

multinomial.h

Description: Computes $\binom{k_1+\cdots+k_n}{k_1,k_2,\ldots,k_n} = \frac{(\sum k_i)!}{k_1!k_2!\ldots k_n!}$.

```
ll multinomial(vi& v) {
  ll c = 1, m = v.empty() ? 1 : v[0];
  rep(i,1,sz(v)) rep(j,0,v[i]) c = c * ++m / (j+1);
  return c;
}
```

4.3 General purpose numbers

4.3.1 Bernoulli numbers

EGF of Bernoulli numbers is $B(t) = \frac{t}{e^t-1}$ (FFT-able).
$B[0,\ldots] = [1, -\frac{1}{2}, \frac{1}{6}, 0, -\frac{1}{30}, 0, \frac{1}{42}, \ldots]$

Sums of powers:

$$\sum_{i=1}^{n} n^m = \frac{1}{m+1}\sum_{k=0}^{m}\binom{m+1}{k}B_k \cdot (n+1)^{m+1-k}$$

Euler-Maclaurin formula for infinite sums:

$$\sum_{i=m}^{\infty} f(i) = \int_m^{\infty} f(x)dx - \sum_{k=1}^{\infty}\frac{B_k}{k!}f^{(k-1)}(m)$$

$$\approx \int_m^{\infty} f(x)dx + \frac{f(m)}{2} - \frac{f'(m)}{12} + \frac{f'''(m)}{720} + O(f^{(5)}(m))$$

4.3.2 Stirling numbers of the first kind

Number of permutations on n items with k cycles.

$$c(n,k) = c(n-1,k-1) + (n-1)c(n-1,k),\ c(0,0) = 1$$
$$\textstyle\sum_{k=0}^{n} c(n,k)x^k = x(x+1)\ldots(x+n-1)$$

$c(8,k) = 8, 0, 5040, 13068, 13132, 6769, 1960, 322, 28, 1$
$c(n,2) = 0, 0, 1, 3, 11, 50, 274, 1764, 13068, 109584, \ldots$

4.3.3 Eulerian numbers

Number of permutations $\pi \in S_n$ in which exactly k elements are greater than the previous element. k j:s s.t. $\pi(j) > \pi(j+1)$, $k+1$ j:s s.t. $\pi(j) \geq j$, k j:s s.t. $\pi(j) > j$.

$$E(n,k) = (n-k)E(n-1,k-1) + (k+1)E(n-1,k)$$
$$E(n,0) = E(n,n-1) = 1$$
$$E(n,k) = \sum_{j=0}^{k}(-1)^j\binom{n+1}{j}(k+1-j)^n$$

4.3.4 Stirling numbers of the second kind

Partitions of n distinct elements into exactly k groups.

$$S(n,k) = S(n-1,k-1) + kS(n-1,k)$$
$$S(n,1) = S(n,n) = 1$$
$$S(n,k) = \frac{1}{k!}\sum_{j=0}^{k}(-1)^{k-j}\binom{k}{j}j^n$$

4.3.5 Bell numbers

Total number of partitions of n distinct elements. $B(n) = 1, 1, 2, 5, 15, 52, 203, 877, 4140, 21147, \ldots$. For p prime,

$$B(p^m + n) \equiv mB(n) + B(n+1) \pmod p$$

4.3.6 Labeled unrooted trees

on n vertices: n^{n-2}
on k existing trees of size n_i: $n_1 n_2 \cdots n_k n^{k-2}$
with degrees d_i: $(n-2)!/((d_1-1)!\cdots(d_n-1)!)$

4.3.7 Catalan numbers

$$C_n = \frac{1}{n+1}\binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n+1} = \frac{(2n)!}{(n+1)!n!}$$

$$C_0 = 1,\ C_{n+1} = \frac{2(2n+1)}{n+2}C_n,\ C_{n+1} = \sum C_i C_{n-i}$$

$C_n = 1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796, 58786, \ldots$

- sub-diagonal monotone paths in an $n \times n$ grid.
- strings with n pairs of parenthesis, correctly nested.
- binary trees with with $n+1$ leaves (0 or 2 children).
- ordered trees with $n+1$ vertices.
- ways a convex polygon with $n+2$ sides can be cut into triangles by connecting vertices with straight lines.
- permutations of $[n]$ with no 3-term increasing subseq.

Numerical (5)

5.1 Polynomials and recurrences

Polynomial.h

```
struct Poly {
  vector<double> a;
  double operator()(double x) const {
    double val = 0;
    for (int i = sz(a); i--;) (val *= x) += a[i];
    return val;
  }
  void diff() {
    rep(i,1,sz(a)) a[i-1] = i*a[i];
    a.pop_back();
  }
  void divroot(double x0) {
    double b = a.back(), c; a.back() = 0;
    for(int i=sz(a)-1; i--;) c = a[i], a[i] = a[i+1]*x0+b, b=c;
    a.pop_back();
  }
};
```

PolyRoots.h

Description: Finds the real roots to a polynomial.
Usage: `polyRoots({{2,-3,1}},-1e9,1e9) // solve x^2-3x+2 = 0`
Time: $\mathcal{O}\left(n^2 \log(1/\epsilon)\right)$

"Polynomial.h"

```
vector<double> polyRoots(Poly p, double xmin, double xmax) {
  if (sz(p.a) == 2) { return {-p.a[0]/p.a[1]}; }
  vector<double> ret;
  Poly der = p;
  der.diff();
  auto dr = polyRoots(der, xmin, xmax);
  dr.push_back(xmin-1);
  dr.push_back(xmax+1);
  sort(all(dr));
  rep(i,0,sz(dr)-1) {
    double l = dr[i], h = dr[i+1];
    bool sign = p(l) > 0;
    if (sign ^ (p(h) > 0)) {
      rep(it,0,60) { // while (h - l > 1e-8)
        double m = (l + h) / 2, f = p(m);
        if ((f <= 0) ^ sign) l = m;
        else h = m;
      }
      ret.push_back((l + h) / 2);
    }
  }
  return ret;
}
```

PolyInterpolate.h

Description: Given n points (x[i], y[i]), computes an n-1-degree polynomial p that passes through them: $p(x) = a[0] * x^0 + ... + a[n-1] * x^{n-1}$. For numerical precision, pick $x[k] = c * \cos(k/(n-1) * \pi), k = 0 \dots n-1$.
Time: $\mathcal{O}\left(n^2\right)$

```
typedef vector<double> vd;
vd interpolate(vd x, vd y, int n) {
  vd res(n), temp(n);
  rep(k,0,n-1) rep(i,k+1,n)
    y[i] = (y[i] - y[k]) / (x[i] - x[k]);
  double last = 0; temp[0] = 1;
  rep(k,0,n) rep(i,0,n) {
    res[i] += y[k] * temp[i];
    swap(last, temp[i]);
    temp[i] -= last * x[k];
  }
  return res;
}
```

BerlekampMassey.h

Description: Recovers any n-order linear recurrence relation from the first $2n$ terms of the recurrence. Useful for guessing linear recurrences after brute-forcing the first terms. Should work on any field, but numerical stability for floats is not guaranteed. Output will have size $\leq n$.
Usage: `berlekampMassey({0, 1, 1, 3, 5, 11}) // {1, 2}`
Time: $\mathcal{O}\left(N^2\right)$

"../number-theory/ModPow.h"

```
vector<ll> berlekampMassey(vector<ll> s) {
  int n = sz(s), L = 0, m = 0;
  vector<ll> C(n), B(n), T;
  C[0] = B[0] = 1;

  ll b = 1;
  rep(i,0,n) { ++m;
    ll d = s[i] % mod;
    rep(j,1,L+1) d = (d + C[j] * s[i - j]) % mod;
    if (!d) continue;
    T = C; ll coef = d * modpow(b, mod-2) % mod;
    rep(j,m,n) C[j] = (C[j] - coef * B[j - m]) % mod;
    if (2 * L > i) continue;
    L = i + 1 - L; B = T; b = d; m = 0;
  }

  C.resize(L + 1); C.erase(C.begin());
  for (ll& x : C) x = (mod - x) % mod;
  return C;
}
```

LinearRecurrence.h

Description: Generates the k'th term of an n-order linear recurrence $S[i] = \sum_j S[i-j-1]tr[j]$, given $S[0 \dots \geq n-1]$ and $tr[0 \dots n-1]$. Faster than matrix multiplication. Useful together with Berlekamp–Massey.
Usage: `linearRec({0, 1}, {1, 1}, k) // k'th Fibonacci number`
Time: $\mathcal{O}\left(n^2 \log k\right)$

```
typedef vector<ll> Poly;
ll linearRec(Poly S, Poly tr, ll k) {
  int n = sz(tr);

  auto combine = [&](Poly a, Poly b) {
    Poly res(n * 2 + 1);
    rep(i,0,n+1) rep(j,0,n+1)
      res[i + j] = (res[i + j] + a[i] * b[j]) % mod;
    for (int i = 2 * n; i > n; --i) rep(j,0,n)
      res[i - 1 - j] = (res[i - 1 - j] + res[i] * tr[j]) % mod;
    res.resize(n + 1);
    return res;
  };

  Poly pol(n + 1), e(pol);
  pol[0] = e[1] = 1;

  for (++k; k; k /= 2) {
    if (k % 2) pol = combine(pol, e);
    e = combine(e, e);
  }

  ll res = 0;
  rep(i,0,n) res = (res + pol[i + 1] * S[i]) % mod;
  return res;
}
```

5.2 Matrices

Determinant.h

Description: Calculates determinant of a matrix. Destroys the matrix.
Time: $\mathcal{O}\left(N^3\right)$

```
double det(vector<vector<double>>& a) {
  int n = sz(a); double res = 1;
  rep(i,0,n) {
    int b = i;
    rep(j,i+1,n) if (fabs(a[j][i]) > fabs(a[b][i])) b = j;
    if (i != b) swap(a[i], a[b]), res *= -1;
    res *= a[i][i];
    if (res == 0) return 0;
    rep(j,i+1,n) {
      double v = a[j][i] / a[i][i];
      if (v != 0) rep(k,i+1,n) a[j][k] -= v * a[i][k];
    }
  }
  return res;
}
```

IntDeterminant.h

Description: Calculates determinant using modular arithmetics. Modulos can also be removed to get a pure-integer version.
Time: $\mathcal{O}\left(N^3\right)$

```
const ll mod = 12345;
ll det(vector<vector<ll>>& a) {
  int n = sz(a); ll ans = 1;
  rep(i,0,n) {
    rep(j,i+1,n) {
      while (a[j][i] != 0) { // gcd step
        ll t = a[i][i] / a[j][i];
        if (t) rep(k,i,n)
          a[i][k] = (a[i][k] - a[j][k] * t) % mod;
        swap(a[i], a[j]);
        ans *= -1;
      }
    }
    ans = ans * a[i][i] % mod;
    if (!ans) return 0;
  }
  return (ans + mod) % mod;
}
```

SolveLinearMod.h

Description: Solves A*x = b mod p (p prime). Input a is the augmented matrix [A — b] with entries in [0, mod). Returns rank, or -1 if no solution. If multiple solutions, one arbitrary solution is returned. If $b = 0$, returns non-zero solution if it exists (rank < m).
Time: $\mathcal{O}\left(n^2 m\right)$

```
// requires: const int mod (prime); int power(int, int)
int Gauss(vector<vector<int>> a, vector<int>& ans) {
  int n = a.size();
  if (n == 0) { ans.clear(); return 0; }
  int m = (int)a[0].size() - 1;
  const ll MODSQ = (ll)mod * mod;
  vector<int> pos(m, -1);
  int rank = 0;

  for (int col = 0, row = 0; col < m && row < n; col++) {
    int mx = -1;
    rep(k,row,n) if(a[k][col]) {mx = k; break;}
    if (mx == -1) continue;
    swap(a[mx], a[row]);
    pos[col] = row;
    int inv = power(a[row][col], mod - 2);
    rep(i,0,n) {
      if (i != row && a[i][col]) {
        ll x = (ll)a[i][col] * inv % mod;
        for (int j = col; j <= m; j++)
          a[i][j] = (MODSQ + a[i][j] - a[row][j] * x) % mod;
      }
```

```
    }
    row++; rank++;
  }

  rep(i,rank,n) if (a[i][m]) return -1;       // inconsistent

  ans.assign(m, 0);
  int f = -1;
  rep(i,0,m) if(pos[i] == -1) {f=i; break;}
  if (f != -1) ans[f] = 1;              // first free var = 1 (
       nonzero if rank < m)

  rep(i,0,m) {
    if (pos[i] == -1) continue;
    long long v = a[pos[i]][m];
    if (f != -1) v = (v - 1LL * a[pos[i]][f] * ans[f]) % mod;
    v = (v + mod) % mod;
    ans[i] = v * power(a[pos[i]][i], mod - 2) % mod;
  }
  return rank;
}
```

SolveLinear.h

Description: Solves $A * x = b$. If there are multiple solutions, an arbitrary one is returned. If $b = 0$, returns non-zero solution if it exists (rank < m). Returns rank, or -1 if no solutions. Data in A and b is lost.
Time: $\mathcal{O}\left(n^2 m\right)$

```
typedef vector<double> vd;
const double eps = 1e-12;

int solveLinear(vector<vd>& A, vd& b, vd& x) {
  int n = sz(A), m = sz(x), rank = 0, br, bc;
  if (n) assert(sz(A[0]) == m);
  vi col(m); iota(all(col), 0);

  rep(i,0,n) {
    double v, bv = 0;
    rep(r,i,n) rep(c,i,m)
      if ((v = fabs(A[r][c])) > bv)
        br = r, bc = c, bv = v;
    if (bv <= eps) {
      rep(j,i,n) if (fabs(b[j]) > eps) return -1;
      break;
    }
    swap(A[i], A[br]);
    swap(b[i], b[br]);
    swap(col[i], col[bc]);
    rep(j,0,n) swap(A[j][i], A[j][bc]);
    bv = 1/A[i][i];
    rep(j,i+1,n) {
      double fac = A[j][i] * bv;
      b[j] -= fac * b[i];
      rep(k,i+1,m) A[j][k] -= fac*A[i][k];
    }
    rank++;
  }

  x.assign(m, 0);
  if (rank < m) x[col[rank]] = 1;
  for (int i = rank; i--;) {
    rep(j,i+1,m) b[i] -= A[i][j] * x[col[j]];
    x[col[i]] = b[i] / A[i][i];
  }
  return rank; // (multiple solutions if rank < m)
}
```

SolveLinear2.h

Description: To get all uniquely determined values of x back from SolveLinear, make the following changes:

"SolveLinear.h"

```
rep(j,0,n) if (j != i) // instead of rep(j,i+1,n)
// ... then at the end:
x.assign(m, undefined);
rep(i,0,rank) {
  rep(j,rank,m) if (fabs(A[i][j]) > eps) goto fail;
  x[col[i]] = b[i] / A[i][i];
fail:; }
```

SolveLinearMod2.h

Description: Solves $Ax = b$ over $\mathbb{F}_2$. If there are multiple solutions, one is returned arbitrarily. If $b = 0$, returns non-zero solution if it exists (rank < m). Returns rank, or -1 if no solutions. Destroys A and b.
Time: $\mathcal{O}\left(n^2 m\right)$

```
typedef bitset<1500> bs; // max. no. of variables
int solveLinear(vector<bs>& A, vi& b, bs& x, int m) {
  int n = sz(A), rank = 0, br;
  assert(m <= sz(x));
  vi col(m); iota(all(col), 0);
  rep(i,0,n) {
    for (br=i; br<n; ++br) if (A[br].any()) break;
    if (br == n) {
      rep(j,i,n) if(b[j]) return -1;
      break;
    }
    int bc = (int)A[br]._Find_next(i-1);
    swap(A[i], A[br]);
    swap(b[i], b[br]);
    swap(col[i], col[bc]);
    rep(j,0,n) if (A[j][i] != A[j][bc]) {
      A[j].flip(i); A[j].flip(bc);
    }
    rep(j,i+1,n) if (A[j][i]) {
      b[j] ^= b[i];
      A[j] ^= A[i];
    }
    rank++;
  }
  x = bs();
  if (rank < m) x[col[rank]] = 1;
  for (int i = rank - 1; i >= 0; --i) {
      int v = b[i];
      for(int j = i+1; j<m; ++j) v ^= (A[i][j] & x[col[j]]);
      x[col[i]] = v;
    }
    return rank;
}
```

XorBasis.h

```
struct XorBasis {
    static const int B = 60;      // for 64-bit numbers
    long long b[B];  int sz;
    XorBasis() {
        memset(b, 0, sizeof b);  sz = 0;
    }
    // Insert number into basis
    void insert(long long x) {
        for (int i = B-1; i >= 0; i--) {
            if (!(x >> i & 1)) continue;
            if (b[i]) x ^= b[i];         // eliminate bit
            else {                       // new basis vector
                b[i] = x;  sz++;  return;
            }}}
    // Check if some subset has xor = x
    bool can(long long x) {
        for(int i = B-1; i>=0; i--) x = min(x, x ^ b[i]);
        return x == 0;
    }
    // Maximum possible xor with optional start value
    long long max_xor(long long x = 0) {
        for(int i = B-1; i>=0; i--) x = max(x, x ^ b[i]);
        return x;
    }
    // k-th smallest subset xor (1-indexed, 1st = 0)
    long long kth(long long k) {
        if (k < 1 || k > (1LL << sz)) return -1;
        long long x = 0, cnt = 1LL << sz;
        for (int i = B-1; i >= 0; i--) if (b[i]) {
            if (k > cnt/2) {
                if (!(x >> i & 1)) x ^= b[i];  k -= cnt/2;
            } else {
                if (x >> i & 1) x ^= b[i];
            }
            cnt /= 2;
        }
        return x;
    }
    // Count how many subset xors are < x
    long long count_lt(long long x) {
        if (x < 0) return 0;
        long long ans = 0, cnt = 1LL << sz;
        long long mask = 0;
        for (int i = B-1; i >= 0; i--) {
            if (b[i]) {
                if (x >> i & 1) {
                    ans += cnt/2;
                    if (!(mask >> i & 1)) mask ^= b[i];
                } else if (mask >> i & 1) {
                    mask ^= b[i];
                }
                cnt /= 2;
            }
            else if ((x >> i & 1) != (mask >> i & 1)) {
                return (x >> i & 1) ? ans + cnt : ans;
            }
        }
        return ans;
    }};
```

MatrixInverse.h

Description: Invert matrix A. Returns rank; result is stored in A unless singular (rank < n). Can easily be extended to prime moduli; for prime powers, repeatedly set $A^{-1} = A^{-1}(2I - AA^{-1}) \pmod{p^k}$ where A^{-1} starts as the inverse of A mod p, and k is doubled in each step.
Time: $\mathcal{O}\left(n^3\right)$

```
int matInv(vector<vector<double>>& A) {
  int n = sz(A); vi col(n);
  vector<vector<double>> tmp(n, vector<double>(n));
  rep(i,0,n) tmp[i][i] = 1, col[i] = i;

  rep(i,0,n) {
    int r = i, c = i;
    rep(j,i,n) rep(k,i,n)
      if (fabs(A[j][k]) > fabs(A[r][c]))
        r = j, c = k;
    if (fabs(A[r][c]) < 1e-12) return i;
    A[i].swap(A[r]); tmp[i].swap(tmp[r]);
    rep(j,0,n)
      swap(A[j][i], A[j][c]), swap(tmp[j][i], tmp[j][c]);
    swap(col[i], col[c]);
    double v = A[i][i];
    rep(j,i+1,n) {
      double f = A[j][i] / v;
```

```
      A[j][i] = 0;
      rep(k,i+1,n) A[j][k] -= f*A[i][k];
      rep(k,0,n) tmp[j][k] -= f*tmp[i][k];
    }
    rep(j,i+1,n) A[i][j] /= v;
    rep(j,0,n) tmp[i][j] /= v;
    A[i][i] = 1;
  }

  for (int i = n-1; i > 0; --i) rep(j,0,i) {
    double v = A[j][i];
    rep(k,0,n) tmp[j][k] -= v*tmp[i][k];
  }

  rep(i,0,n) rep(j,0,n) A[col[i]][col[j]] = tmp[i][j];
  return n;
}
```

MatrixInverse-mod.h

Description: Invert matrix A modulo a prime. Returns rank; result is stored in A unless singular (rank < n). For prime powers, repeatedly set $A^{-1} = A^{-1}(2I - AA^{-1})$ (mod p^k) where A^{-1} starts as the inverse of A mod p, and k is doubled in each step.
Time: $\mathcal{O}(n^3)$

"../number-theory/ModPow.h"

```
int matInv(vector<vector<ll>>& A) {
  int n = sz(A); vi col(n);
  vector<vector<ll>> tmp(n, vector<ll>(n));
  rep(i,0,n) tmp[i][i] = 1, col[i] = i;

  rep(i,0,n) {
    int r = i, c = i;
    rep(j,i,n) rep(k,i,n) if (A[j][k]) {
      r = j; c = k; goto found;
    }
    return i;
found:
    A[i].swap(A[r]); tmp[i].swap(tmp[r]);
    rep(j,0,n)
      swap(A[j][i], A[j][c]), swap(tmp[j][i], tmp[j][c]);
    swap(col[i], col[c]);
    ll v = modpow(A[i][i], mod - 2);
    rep(j,i+1,n) {
      ll f = A[j][i] * v % mod;
      A[j][i] = 0;
      rep(k,i+1,n) A[j][k] = (A[j][k] - f*A[i][k]) % mod;
      rep(k,0,n) tmp[j][k] = (tmp[j][k] - f*tmp[i][k]) % mod;
    }
    rep(j,i+1,n) A[i][j] = A[i][j] * v % mod;
    rep(j,0,n) tmp[i][j] = tmp[i][j] * v % mod;
    A[i][i] = 1;
  }

  for (int i = n-1; i > 0; --i) rep(j,0,i) {
    ll v = A[j][i];
    rep(k,0,n) tmp[j][k] = (tmp[j][k] - v*tmp[i][k]) % mod;
  }

  rep(i,0,n) rep(j,0,n)
    A[col[i]][col[j]] = tmp[i][j] % mod + (tmp[i][j] < 0)*mod;
  return n;
}
```

5.3 Fourier transforms

FastFourierTransform.h

Description: fft(a) computes $\hat{f}(k) = \sum_x a[x] \exp(2\pi i \cdot kx/N)$ for all k. N must be a power of 2. Useful for convolution: `conv(a, b) = c`, where $c[x] = \sum a[i]b[x-i]$. For convolution of complex numbers or more than two vectors: FFT, multiply pointwise, divide by n, reverse(start+1, end), FFT back. Rounding is safe if $(\sum a_i^2 + \sum b_i^2)\log_2 N < 9 \cdot 10^{14}$ (in practice 10^{16}; higher for random inputs). Otherwise, use NTT/FFTMod.
Time: $\mathcal{O}(N \log N)$ with $N = |A| + |B|$ ($\sim 1s$ for $N = 2^{22}$)

```
typedef complex<double> C;
typedef vector<double> vd;
void fft(vector<C>& a) {
  int n = sz(a), L = 31 - __builtin_clz(n);
  static vector<complex<long double>> R(2, 1);
  static vector<C> rt(2, 1);  // (^ 10% faster if double)
  for (static int k = 2; k < n; k *= 2) {
    R.resize(n); rt.resize(n);
    auto x = polar(1.0L, acos(-1.0L) / k);
    rep(i,k,2*k) rt[i] = R[i] = i&1 ? R[i/2] * x : R[i/2];
  }
  vi rev(n);
  rep(i,0,n) rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
  rep(i,0,n) if (i < rev[i]) swap(a[i], a[rev[i]]);
  for (int k = 1; k < n; k *= 2)
    for (int i = 0; i < n; i += 2 * k) rep(j,0,k) {
      C z = rt[j+k] * a[i+j+k]; // (25% faster if hand-rolled)
      a[i + j + k] = a[i + j] - z;
      a[i + j] += z;
    }
}
vd conv(const vd& a, const vd& b) {
  if (a.empty() || b.empty()) return {};
  vd res(sz(a) + sz(b) - 1);
  int L = 32 - __builtin_clz(sz(res)), n = 1 << L;
  vector<C> in(n), out(n);
  copy(all(a), begin(in));
  rep(i,0,sz(b)) in[i].imag(b[i]);
  fft(in);
  for (C& x : in) x *= x;
  rep(i,0,n) out[i] = in[-i & (n - 1)] - conj(in[i]);
  fft(out);
  rep(i,0,sz(res)) res[i] = imag(out[i]) / (4 * n);
  return res;
}
```

FastFourierTransformMod.h

Description: Higher precision FFT, can be used for convolutions modulo arbitrary integers as long as $N \log_2 N \cdot \text{mod} < 8.6 \cdot 10^{14}$ (in practice 10^{16} or higher). Inputs must be in $[0, \text{mod})$.
Time: $\mathcal{O}(N \log N)$, where $N = |A| + |B|$ (twice as slow as NTT or FFT)

"FastFourierTransform.h"

```
typedef vector<ll> vl;
template<int M> vl convMod(const vl &a, const vl &b) {
  if (a.empty() || b.empty()) return {};
  vl res(sz(a) + sz(b) - 1);
  int B=32-__builtin_clz(sz(res)), n=1<<B, cut=int(sqrt(M));
  vector<C> L(n), R(n), outs(n), outl(n);
  rep(i,0,sz(a)) L[i] = C((int)a[i] / cut, (int)a[i] % cut);
  rep(i,0,sz(b)) R[i] = C((int)b[i] / cut, (int)b[i] % cut);
  fft(L), fft(R);
  rep(i,0,n) {
    int j = -i & (n - 1);
    outl[j] = (L[i] + conj(L[j])) * R[i] / (2.0 * n);
    outs[j] = (L[i] - conj(L[j])) * R[i] / (2.0 * n) / 1i;
  }
  fft(outl), fft(outs);
  rep(i,0,sz(res)) {
    ll av = ll(real(outl[i])+.5), cv = ll(imag(outs[i])+.5);
    ll bv = ll(imag(outl[i])+.5) + ll(real(outs[i])+.5);
    res[i] = ((av % M * cut + bv) % M * cut + cv) % M;
  }
  return res;
}
```

NumberTheoreticTransform.h

Description: ntt(a) computes $\hat{f}(k) = \sum_x a[x] g^{xk}$ for all k, where $g = \text{root}^{(mod-1)/N}$. N must be a power of 2. Useful for convolution modulo specific nice primes of the form $2^a b + 1$, where the convolution result has size at most 2^a. For arbitrary modulo, see FFTMod. `conv(a, b) = c`, where $c[x] = \sum a[i]b[x-i]$. For manual convolution: NTT the inputs, multiply pointwise, divide by n, reverse(start+1, end), NTT back. Inputs must be in [0, mod).
Time: $\mathcal{O}(N \log N)$

"../number-theory/ModPow.h"

```
const ll mod = (119 << 23) + 1, root = 62; // = 998244353

typedef vector<ll> vl;
void ntt(vl &a) {
  int n = sz(a), L = 31 - __builtin_clz(n);
  static vl rt(2, 1);
  for (static int k = 2, s = 2; k < n; k *= 2, s++) {
    rt.resize(n);
    ll z[] = {1, modpow(root, mod >> s)};
    rep(i,k,2*k) rt[i] = rt[i / 2] * z[i & 1] % mod;
  }
  vi rev(n);
  rep(i,0,n) rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
  rep(i,0,n) if (i < rev[i]) swap(a[i], a[rev[i]]);
  for (int k = 1; k < n; k *= 2)
    for (int i = 0; i < n; i += 2 * k) rep(j,0,k) {
      ll z = rt[j + k] * a[i + j + k] % mod, &ai = a[i + j];
      a[i + j + k] = ai - z + (z > ai ? mod : 0);
      ai += (ai + z >= mod ? z - mod : z);
    }
}
vl conv(const vl &a, const vl &b) {
  if (a.empty() || b.empty()) return {};
  int s = sz(a) + sz(b) - 1, B = 32 - __builtin_clz(s),
      n = 1 << B;
  int inv = modpow(n, mod - 2);
  vl L(a), R(b), out(n);
  L.resize(n), R.resize(n);
  ntt(L), ntt(R);
  rep(i,0,n)
    out[-i & (n - 1)] = (ll)L[i] * R[i] % mod * inv % mod;
  ntt(out);
  return {out.begin(), out.begin() + s};
}
```

FastSubsetTransform.h

Description: Transform to a basis with fast convolutions of the form $c[z] = \sum_{z=x \oplus y} a[x] \cdot b[y]$, where $\oplus$ is one of AND, OR, XOR. The size of a must be a power of two.
Time: $\mathcal{O}(N \log N)$

```
void FST(vi& a, bool inv) {
  for (int n = sz(a), step = 1; step < n; step *= 2) {
    for (int i = 0; i < n; i += 2 * step) rep(j,i,i+step) {
      int &u = a[j], &v = a[j + step]; tie(u, v) =
        inv ? pii(v - u, u) : pii(v, u + v); // AND
        inv ? pii(v, u - v) : pii(u + v, u); // OR
        pii(u + v, u - v);                   // XOR
    }
  }
  if (inv) for (int& x : a) x /= sz(a); // XOR only
}
vi conv(vi a, vi b) {
```

```
  FST(a, 0); FST(b, 0);
  rep(i,0,sz(a)) a[i] *= b[i];
  FST(a, 1); return a;
}
```

Graph (6)

6.1 Fundamentals

Dijkstra.h

```
vector<ll> dijkstra(ll start, vector<vector<array<ll, 2>>> &gr,
      int n) {
    vector<ll> d(n + 1, inf);
    d[start] = 0;

    priority_queue<array<ll, 2>, vector<array<ll, 2>>, greater<
        array<ll, 2>>> pq;
    pq.push({0LL, start});

    while (!pq.empty()) {
        auto [dist, node] = pq.top();
        pq.pop();

        if (dist > d[node]) continue;

        for (auto [nextNode, weight] : gr[node]) {
            if (weight + dist < d[nextNode]) {
                d[nextNode] = weight + dist;
                pq.push({d[nextNode], nextNode});
            }
        }
    }
    return d;
}
```

BellmanFord.h

Description: Calculates shortest paths from s in a graph that might have negative edge weights. Unreachable nodes get dist = inf; nodes reachable through negative-weight cycles get dist = -inf. Assumes $V^2 \max|w_i| < \sim 2^{63}$.
Time: $\mathcal{O}(VE)$

```
const ll inf = LLONG_MAX;
struct Ed { int a, b, w, s() { return a < b ? a : -a; }};
struct Node { ll dist = inf; int prev = -1; };

void bellmanFord(vector<Node>& nodes, vector<Ed>& eds, int s) {
  nodes[s].dist = 0;
  sort(all(eds), [](Ed a, Ed b) { return a.s() < b.s(); });

  int lim = sz(nodes) / 2 + 2; // /3+100 with shuffled vertices
  rep(i,0,lim) for (Ed ed : eds) {
    Node cur = nodes[ed.a], &dest = nodes[ed.b];
    if (abs(cur.dist) == inf) continue;
    ll d = cur.dist + ed.w;
    if (d < dest.dist) {
      dest.prev = ed.a;
      dest.dist = (i < lim-1 ? d : -inf);
    }
  }
  rep(i,0,lim) for (Ed e : eds) {
    if (nodes[e.a].dist == -inf)
      nodes[e.b].dist = -inf;
  }
}
```

FloydWarshall.h

Description: Calculates all-pairs shortest path in a directed graph that might have negative edge weights. Input is an distance matrix m, where $m[i][j] =$ inf if i and j are not adjacent. As output, $m[i][j]$ is set to the shortest distance between i and j, inf if no path, or -inf if the path goes through a negative-weight cycle.
Time: $\mathcal{O}(N^3)$

```
const ll inf = 1LL << 62;
void floydWarshall(vector<vector<ll>>& m) {
  int n = sz(m);
  rep(i,0,n) m[i][i] = min(m[i][i], 0LL);
  rep(k,0,n) rep(i,0,n) rep(j,0,n)
    if (m[i][k] != inf && m[k][j] != inf) {
      auto newDist = max(m[i][k] + m[k][j], -inf);
      m[i][j] = min(m[i][j], newDist);
    }
  rep(k,0,n) if (m[k][k] < 0) rep(i,0,n) rep(j,0,n)
    if (m[i][k] != inf && m[k][j] != inf) m[i][j] = -inf;
}
```

TopoSort.h

Description: Topological sorting. Given is an oriented graph. Output is an ordering of vertices, such that there are edges only from left to right. If there are cycles, the returned list will have size smaller than n – nodes reachable from cycles will not be returned.
Time: $\mathcal{O}(|V| + |E|)$

```
vi topoSort(const vector<vi>& gr) {
  vi indeg(sz(gr)), q;
  for (auto& li : gr) for (int x : li) indeg[x]++;
  rep(i,0,sz(gr)) if (indeg[i] == 0) q.push_back(i);
  rep(j,0,sz(q)) for (int x : gr[q[j]])
    if (--indeg[x] == 0) q.push_back(x);
  return q;
}
```

TravellingSalesman.h

```
// cost[u][v] = edge weight, INF if no edge
// O(n^2 * 2^n)
ll tsp(vector<vector<ll>>& cost) {
    int n = cost.size();
    vector dp(1 << n, vector<ll>(n, inf));
    dp[1][0] = 0; // start at 0

    for (int mask = 0; mask < (1 << n); mask++) {
        if (!(mask & 1)) continue;

        for (int u = 0; u < n; u++) {
            if (dp[mask][u] == inf) continue;

            for (int v = 0; v < n; v++) {
                if (mask & (1 << v)) continue;
                if (cost[u][v] == inf) continue;
                dp[mask | (1 << v)][v] =
                    min(dp[mask | (1 << v)][v],
                        dp[mask][u] + cost[u][v]);
            }
        }
    }
    ll ans = inf;
    int full = (1 << n) - 1;
    for (int u = 0; u < n; u++) {
        if (cost[u][0] == inf) continue;
        ans = min(ans, dp[full][u] + cost[u][0]);
    }
    return ans;
}
```

6.2 Network flow

PushRelabel.h

Description: Push-relabel using the highest label selection rule and the gap heuristic. Quite fast in practice. To obtain the actual flow, look at positive values only.
Time: $\mathcal{O}\left(V^2\sqrt{E}\right)$

```
struct PushRelabel {
  struct Edge {
    int dest, back;
    ll f, c;
  };
  vector<vector<Edge>> g;
  vector<ll> ec;
  vector<Edge*> cur;
  vector<vi> hs; vi H;
  PushRelabel(int n) : g(n), ec(n), cur(n), hs(2*n), H(n) {}

  void addEdge(int s, int t, ll cap, ll rcap=0) {
    if (s == t) return;
    g[s].push_back({t, sz(g[t]), 0, cap});
    g[t].push_back({s, sz(g[s])-1, 0, rcap});
  }

  void addFlow(Edge& e, ll f) {
    Edge &back = g[e.dest][e.back];
    if (!ec[e.dest] && f) hs[H[e.dest]].push_back(e.dest);
    e.f += f; e.c -= f; ec[e.dest] += f;
    back.f -= f; back.c += f; ec[back.dest] -= f;
  }
  ll calc(int s, int t) {
    int v = sz(g); H[s] = v; ec[t] = 1;
    vi co(2*v); co[0] = v-1;
    rep(i,0,v) cur[i] = g[i].data();
    for (Edge& e : g[s]) addFlow(e, e.c);

    for (int hi = 0;;) {
      while (hs[hi].empty()) if (!hi--) return -ec[s];
      int u = hs[hi].back(); hs[hi].pop_back();
      while (ec[u] > 0)  // discharge u
        if (cur[u] == g[u].data() + sz(g[u])) {
          H[u] = 1e9;
          for (Edge& e : g[u]) if (e.c && H[u] > H[e.dest]+1)
            H[u] = H[e.dest]+1, cur[u] = &e;
          if (++co[H[u]], !--co[hi] && hi < v)
            rep(i,0,v) if (hi < H[i] && H[i] < v)
              --co[H[i]], H[i] = v + 1;
          hi = H[u];
        } else if (cur[u]->c && H[u] == H[cur[u]->dest]+1)
          addFlow(*cur[u], min(ec[u], cur[u]->c));
        else ++cur[u];
    }
  }
  bool leftOfMinCut(int a) { return H[a] >= sz(g); }
};
```

MinCostMaxFlow.h

Description: Min-cost max-flow. If costs can be negative, call setpi before maxflow, but note that negative cost cycles are not supported. To obtain the actual flow, look at positive values only.
Time: $\mathcal{O}(FE\log(V))$ where F is max flow. $\mathcal{O}(VE)$ for setpi.

```
#include <bits/extc++.h>
// #include <ext/pb_ds/priority_queue.hpp> // use if issue
const ll INF = numeric_limits<ll>::max() / 4;
struct MCMF {
  struct edge {
    int from, to, rev;
```

```
  ll cap, cost, flow;
};
int N;
vector<vector<edge>> ed;
vi seen;
vector<ll> dist, pi;
vector<edge*> par;

MCMF(int N) : N(N), ed(N), seen(N), dist(N), pi(N), par(N) {}

void addEdge(int from, int to, ll cap, ll cost) {
  if (from == to) return;
  ed[from].push_back(edge{ from,to,sz(ed[to]),cap,cost,0 });
  ed[to].push_back(edge{ to,from,sz(ed[from])-1,0,-cost,0 });
}

void path(int s) {
  fill(all(seen), 0);
  fill(all(dist), INF);
  dist[s] = 0; ll di;

  __gnu_pbds::priority_queue<pair<ll, int>> q;
  vector<decltype(q)::point_iterator> its(N);
  q.push({ 0, s });

  while (!q.empty()) {
    s = q.top().second; q.pop();
    seen[s] = 1; di = dist[s] + pi[s];
    for (edge& e : ed[s]) if (!seen[e.to]) {
      ll val = di - pi[e.to] + e.cost;
      if (e.cap - e.flow > 0 && val < dist[e.to]) {
        dist[e.to] = val;
        par[e.to] = &e;
        if (its[e.to] == q.end())
          its[e.to] = q.push({ -dist[e.to], e.to });
        else
          q.modify(its[e.to], { -dist[e.to], e.to });
      }
    }
  }
  rep(i,0,N) pi[i] = min(pi[i] + dist[i], INF);
}

pair<ll, ll> maxflow(int s, int t) {
  ll totflow = 0, totcost = 0;
  while (path(s), seen[t]) {
    ll fl = INF;
    for (edge* x = par[t]; x; x = par[x->from])
      fl = min(fl, x->cap - x->flow);

    totflow += fl;
    for (edge* x = par[t]; x; x = par[x->from]) {
      x->flow += fl;
      ed[x->to][x->rev].flow -= fl;
    }
  }
  rep(i,0,N) for(edge& e : ed[i]) totcost += e.cost * e.flow;
  return {totflow, totcost/2};
}

// If some costs can be negative, call this before maxflow:
void setpi(int s) { // (otherwise, leave this out)
  fill(all(pi), INF); pi[s] = 0;
  int it = N, ch = 1; ll v;
  while (ch-- && it--)
    rep(i,0,N) if (pi[i] != INF)
      for (edge& e : ed[i]) if (e.cap)
        if ((v = pi[i] + e.cost) < pi[e.to])
          pi[e.to] = v, ch = 1;
    assert(it >= 0); // negative cost cycle
  }
};
```

Dinic.h

Description: Flow algorithm with complexity $O(VE\log U)$ where $U = \max|\text{cap}|$. $O(\min(E^{1/2}, V^{2/3})E)$ if $U = 1$; $O(\sqrt{V}E)$ for bipartite matching.

```
struct Dinic {
  struct Edge {
    int to, rev;
    ll c, oc;
    ll flow() { return max(oc - c, 0LL); } // if you need flows
  };
  vi lvl, ptr, q;
  vector<vector<Edge>> adj;
  Dinic(int n) : lvl(n), ptr(n), q(n), adj(n) {}
  void addEdge(int a, int b, ll c, ll rcap = 0) {
    adj[a].push_back({b, sz(adj[b]), c, c});
    adj[b].push_back({a, sz(adj[a]) - 1, rcap, rcap});
  }
  ll dfs(int v, int t, ll f) {
    if (v == t || !f) return f;
    for (int& i = ptr[v]; i < sz(adj[v]); i++) {
      Edge& e = adj[v][i];
      if (lvl[e.to] == lvl[v] + 1)
        if (ll p = dfs(e.to, t, min(f, e.c))) {
          e.c -= p, adj[e.to][e.rev].c += p;
          return p;
        }
    }
    return 0;
  }
  ll calc(int s, int t) {
    ll flow = 0; q[0] = s;
    rep(L,0,31) do { // 'int L=30' maybe faster for random data
      lvl = ptr = vi(sz(q));
      int qi = 0, qe = lvl[s] = 1;
      while (qi < qe && !lvl[t]) {
        int v = q[qi++];
        for (Edge e : adj[v])
          if (!lvl[e.to] && e.c >> (30 - L))
            q[qe++] = e.to, lvl[e.to] = lvl[v] + 1;
      }
      while (ll p = dfs(s, t, LLONG_MAX)) flow += p;
    } while (lvl[t]);
    return flow;
  }
  bool leftOfMinCut(int a) { return lvl[a] != 0; }
};
```

MinCut.h

Description: After running max-flow, the left side of a min-cut from s to t is given by all vertices reachable from s, only traversing edges with positive residual capacity.

GlobalMinCut.h

Description: Find a global minimum cut in an undirected graph, as represented by an adjacency matrix.
Time: $\mathcal{O}\left(V^3\right)$

```
pair<int, vi> globalMinCut(vector<vi> mat) {
  pair<int, vi> best = {INT_MAX, {}};
  int n = sz(mat);
  vector<vi> co(n);
  rep(i,0,n) co[i] = {i};
  rep(ph,1,n) {
    vi w = mat[0];
    size_t s = 0, t = 0;
    rep(it,0,n-ph) { // O(V^2) -> O(E log V) with prio. queue
      w[t] = INT_MIN;
      s = t, t = max_element(all(w)) - w.begin();
      rep(i,0,n) w[i] += mat[t][i];
    }
    best = min(best, {w[t] - mat[t][t], co[t]});
    co[s].insert(co[s].end(), all(co[t]));
    rep(i,0,n) mat[s][i] += mat[t][i];
    rep(i,0,n) mat[i][s] = mat[s][i];
    mat[0][t] = INT_MIN;
  }
  return best;
}
```

GomoryHu.h

Description: Given a list of edges representing an undirected flow graph, returns edges of the Gomory-Hu tree. The max flow between any pair of vertices is given by minimum edge weight along the Gomory-Hu tree path.
Time: $\mathcal{O}(V)$ Flow Computations

"PushRelabel.h"

```
typedef array<ll, 3> Edge;
vector<Edge> gomoryHu(int N, vector<Edge> ed) {
  vector<Edge> tree;
  vi par(N);
  rep(i,1,N) {
    PushRelabel D(N); // Dinic also works
    for (Edge t : ed) D.addEdge(t[0], t[1], t[2], t[2]);
    tree.push_back({i, par[i], D.calc(i, par[i])});
    rep(j,i+1,N)
      if (par[j] == par[i] && D.leftOfMinCut(j)) par[j] = i;
  }
  return tree;
}
```

6.3 Matching

hopcroftKarp.h

Description: Fast bipartite matching algorithm. Graph g should be a list of neighbors of the left partition, and $btoa$ should be a vector full of -1's of the same size as the right partition. Returns the size of the matching. $btoa[i]$ will be the match for vertex i on the right side, or -1 if it's not matched.
Usage: `vi btoa(m, -1); hopcroftKarp(g, btoa);`
Time: $\mathcal{O}\left(\sqrt{V}E\right)$

```
bool dfs(int a, int L, vector<vi>& g, vi& btoa, vi& A, vi& B) {
  if (A[a] != L) return 0;
  A[a] = -1;
  for (int b : g[a]) if (B[b] == L + 1) {
    B[b] = 0;
    if (btoa[b] == -1 || dfs(btoa[b], L + 1, g, btoa, A, B))
      return btoa[b] = a, 1;
  }
  return 0;
}
int hopcroftKarp(vector<vi>& g, vi& btoa) {
  int res = 0;
  vi A(g.size()), B(btoa.size()), cur, next;
  for (;;) {
    fill(all(A), 0);
    fill(all(B), 0);
    cur.clear();
    for (int a : btoa) if(a != -1) A[a] = -1;
    rep(a,0,sz(g)) if(A[a] == 0) cur.push_back(a);
    for (int lay = 1;; lay++) {
      bool islast = 0;
      next.clear();
```

```
      for (int a : cur) for (int b : g[a]) {
        if (btoa[b] == -1) {
          B[b] = lay;
          islast = 1;
        }
        else if (btoa[b] != a && !B[b]) {
          B[b] = lay;
          next.push_back(btoa[b]);
        }
      }
      if (islast) break;
      if (next.empty()) return res;
      for (int a : next) A[a] = lay;
      cur.swap(next);
    }
    rep(a,0,sz(g))
      res += dfs(a, 0, g, btoa, A, B);
  }
}
```

DFSMatching.h

Description: Simple bipartite matching algorithm. Graph g should be a list of neighbors of the left partition, and $btoa$ should be a vector full of -1's of the same size as the right partition. Returns the size of the matching. $btoa[i]$ will be the match for vertex i on the right side, or -1 if it's not matched.
Usage: `vi btoa(m, -1); dfsMatching(g, btoa);`
Time: $\mathcal{O}(VE)$

```
bool find(int j, vector<vi>& g, vi& btoa, vi& vis) {
  if (btoa[j] == -1) return 1;
  vis[j] = 1; int di = btoa[j];
  for (int e : g[di])
    if (!vis[e] && find(e, g, btoa, vis)) {
      btoa[e] = di;
      return 1;
    }
  return 0;
}
int dfsMatching(vector<vi>& g, vi& btoa) {
  vi vis;
  rep(i,0,sz(g)) {
    vis.assign(sz(btoa), 0);
    for (int j : g[i])
      if (find(j, g, btoa, vis)) {
        btoa[j] = i;
        break;
      }
  }
  return sz(btoa) - (int)count(all(btoa), -1);
}
```

MinimumVertexCover.h

Description: Finds a minimum vertex cover in a bipartite graph. The size is the same as the size of a maximum matching, and the complement is a maximum independent set.

`"DFSMatching.h"`

```
vi cover(vector<vi>& g, int n, int m) {
  vi match(m, -1);
  int res = dfsMatching(g, match);
  vector<bool> lfound(n, true), seen(m);
  for (int it : match) if (it != -1) lfound[it] = false;
  vi q, cover;
  rep(i,0,n) if (lfound[i]) q.push_back(i);
  while (!q.empty()) {
    int i = q.back(); q.pop_back();
    lfound[i] = 1;
    for (int e : g[i]) if (!seen[e] && match[e] != -1) {
      seen[e] = true;
      q.push_back(match[e]);
    }
  }
  rep(i,0,n) if (!lfound[i]) cover.push_back(i);
  rep(i,0,m) if (seen[i]) cover.push_back(n+i);
  assert(sz(cover) == res);
  return cover;
}
```

WeightedMatching.h

Description: Given a weighted bipartite graph, matches every node on the left with a node on the right such that no nodes are in two matchings and the sum of the edge weights is minimal. Takes cost[N][M], where cost[i][j] = cost for L[i] to be matched with R[j] and returns (min cost, match), where L[i] is matched with R[match[i]]. Negate costs for max cost. Requires $N \leq M$.
Time: $\mathcal{O}\left(N^2M\right)$

```
pair<int, vi> hungarian(const vector<vi> &a) {
  if (a.empty()) return {0, {}};
  int n = sz(a) + 1, m = sz(a[0]) + 1;
  vi u(n), v(m), p(m), ans(n - 1);
  rep(i,1,n) {
    p[0] = i;
    int j0 = 0; // add "dummy" worker 0
    vi dist(m, INT_MAX), pre(m, -1);
    vector<bool> done(m + 1);
    do { // dijkstra
      done[j0] = true;
      int i0 = p[j0], j1, delta = INT_MAX;
      rep(j,1,m) if (!done[j]) {
        auto cur = a[i0 - 1][j - 1] - u[i0] - v[j];
        if (cur < dist[j]) dist[j] = cur, pre[j] = j0;
        if (dist[j] < delta) delta = dist[j], j1 = j;
      }
      rep(j,0,m) {
        if (done[j]) u[p[j]] += delta, v[j] -= delta;
        else dist[j] -= delta;
      }
      j0 = j1;
    } while (p[j0]);
    while (j0) { // update alternating path
      int j1 = pre[j0];
      p[j0] = p[j1], j0 = j1;
    }
  }
  rep(j,1,m) if (p[j]) ans[p[j] - 1] = j - 1;
  return {-v[0], ans}; // min cost
}
```

6.4 DFS algorithms

SCC.h

```
vvll gr(N), revgr(N);
vector<bool> vis(N);
stack<ll> st;
void dfs1(ll node) {
    vis[node] = 1;
    for(ll ch : gr[node]) { if(!vis[ch]) dfs1(ch); }
    st.push(node);
}
void dfs2(ll node, vll &comp) {
    vis[node] = 1; comp.push_back(node);
    for(ll ch : revgr[node]) { if(!vis[ch]) dfs2(ch, comp); }
}
vvll kosaraju() {  // returns comps in topological order
    vvll comps;
    L(i, 1, n) if(!vis[i]) dfs1(i);
    L(i, 1, n) vis[i] = 0;
    while(st.size()) {
        ll now = st.top(); st.pop();
        if(!vis[now]) {
            vll comp; dfs2(now, comp); comps.push_back(comp);
        }
    } return comps;
}
```

BiconnectedComponents.h

Description: Finds all biconnected components in an undirected graph, and runs a callback for the edges in each. In a biconnected component there are at least two internally disjoint paths between any two nodes (a cycle exists through them). Note that a node can be in several components. An edge which is not in a component is a bridge, i.e., not part of any cycle.
Usage: `int eid = 0; ed.resize(N);`
`for each edge (a,b) {`
`ed[a].emplace_back(b, eid);`
`ed[b].emplace_back(a, eid++); }`
`bicomps([&](const vi& edgelist) {...});`
Time: $\mathcal{O}(E+V)$

```
vi num, st;
vector<vector<pii>> ed;
int Time;
template<class F>
int dfs(int at, int par, F& f) {
  int me = num[at] = ++Time, top = me;
  for (auto [y, e] : ed[at]) if (e != par) {
    if (num[y]) {
      top = min(top, num[y]);
      if (num[y] < me)
        st.push_back(e);
    } else {
      int si = sz(st);
      int up = dfs(y, e, f);
      top = min(top, up);
      if (up == me) {
        st.push_back(e);
        f(vi(st.begin() + si, st.end()));
        st.resize(si);
      }
      else if (up < me) st.push_back(e);
      else { /* e is a bridge */ }
    }
  }
  return top;
}
template<class F>
void bicomps(F f) {
  num.assign(sz(ed), 0);
  rep(i,0,sz(ed)) if (!num[i]) dfs(i, -1, f);
}
```

2sat.h

Description: Calculates a valid assignment to boolean variables a, b, c,... to a 2-SAT problem, so that an expression of the type $(a||b)\&\&(!a||c)\&\&(d||!b)\&\&...$ becomes true, or reports that it is unsatisfiable. Negated variables are represented by bit-inversions (~x).
Usage: `TwoSat ts(number of boolean variables);`
`ts.either(0, ~3); // Var 0 is true or var 3 is false`
`ts.setValue(2); // Var 2 is true`
`ts.implies(0, 1); // If Var 0 is true, then Var 1 must be true`
`ts.setXor(0, 1, true); // Var 0 != Var 1`
`ts.atMostOne({0,~1,2}); // <= 1 of vars 0, ~1 and 2 are true`
`ts.solve(); // Returns true iff it is solvable`
`ts.values[0..N-1] holds the assigned values to the vars`
Time: $\mathcal{O}(N+E)$, where N is the number of boolean variables, and E is the number of clauses.

```
struct TwoSat {
  int N;
  vector<vi> gr;
  vi values; // 0 = false, 1 = true
  TwoSat(int n = 0) : N(n), gr(2*n) {}
  int addVar() {
    gr.emplace_back();
    gr.emplace_back();
    return N++;
  }
  void either(int f, int j) {
    f = max(2*f, -1-2*f);
    j = max(2*j, -1-2*j);
    gr[f].push_back(j^1);
    gr[j].push_back(f^1);
  }
  // Force a variable to be true/false: setValue(x) or setValue
       (~x)
  void setValue(int x) { either(x, x); }
  // Implication: if f is true, then j must be true (f => j)
  void implies(int f, int j) { either(~f, j); }
  // if val is false: f^j==0, if val is true:  f^j==1
  void setXor(int f, int j, bool val = true) {
    if (val) {
      either(f, j);
      either(~f, ~j);
    } else {
      either(~f, j);
      either(f, ~j);
    }
  }
  // At most one of the literals in the list can be true
  void atMostOne(const vi& li) {
    if (sz(li) <= 1) return;
    int cur = ~li[0];
    rep(i,2,sz(li)) {
      int next = addVar();
      either(cur, ~li[i]);
      either(cur, next);
      either(~li[i], next);
      cur = ~next;
    }
    either(cur, ~li[1]);
  }

  vi val, comp, z; int time = 0;
  int dfs(int i) {
    int low = val[i] = ++time, x; z.push_back(i);
    for(int e : gr[i]) if (!comp[e])
      low = min(low, val[e] ?: dfs(e));
    if (low == val[i]) do {
      x = z.back(); z.pop_back();
      comp[x] = low;
      if (values[x>>1] == -1)
        values[x>>1] = x&1;
    } while (x != i);
    return val[i] = low;
  }
  bool solve() {
    values.assign(N, -1);
    val.assign(2*N, 0); comp = val;
    rep(i,0,2*N) if (!comp[i]) dfs(i);
    rep(i,0,N) if (comp[2*i] == comp[2*i+1]) return 0;
    return 1;
  }
};
```

EulerWalk.h

Description: Eulerian undirected/directed path/cycle algorithm. Input should be a vector of (dest, global edge index), where for undirected graphs, forward/backward edges have the same index. Returns a list of nodes in the Eulerian path/cycle with src at both start and end, or empty list if no cycle/path exists. To get edge indices back, add .second to s and ret.
Time: $\mathcal{O}(V + E)$

```
vi eulerWalk(vector<vector<pii>>& gr, int nedges, int src=0) {
  int n = sz(gr);
  vi D(n), its(n), eu(nedges), ret, s = {src};
  D[src]++; // to allow Euler paths, not just cycles
  while (!s.empty()) {
    int x = s.back(), y, e, &it = its[x], end = sz(gr[x]);
    if (it == end){ ret.push_back(x); s.pop_back(); continue; }
    tie(y, e) = gr[x][it++];
    if (!eu[e]) {
      D[x]--, D[y]++;
      eu[e] = 1; s.push_back(y);
    }}
  for (int x : D) if (x < 0 || sz(ret) != nedges+1) return {};
  return {ret.rbegin(), ret.rend()};
}
```

BridgesAndArticulationPoints.h

```
struct BridgesAndArtPoint{
 int n,timer=0;
 vector<vector<int>> g;
 vector<int> tin,low;
 vector<char> vis,isArtPoint;
 vector<pair<int,int>> bridges;
 BridgesAndArtPoint(int n=0) : n(n), g(n), tin(n,-1),
  low(n,-1), vis(n,0), isArtPoint(n,0){}

void addEdge(int u,int v){g[u].push_back(v);g[v].push_back(u);}
 void run(){
  timer=0;
  fill(tin.begin(),tin.end(),-1);
  fill(low.begin(),low.end(),-1);
  fill(vis.begin(),vis.end(),0);
  fill(isArtPoint.begin(),isArtPoint.end(),0);
  bridges.clear();
  for(int v=0;v<n;++v) if(!vis[v]) dfs(v,-1);
}
private:
 void dfs(int v,int p){
  vis[v]=1;
  tin[v]=low[v]=timer++;
  int children=0;
  for(int to:g[v]){
   if(to==p) continue;
   if(vis[to]){ low[v]=min(low[v],tin[to]); }
   else{
    ++children; dfs(to,v);
    low[v]=min(low[v],low[to]);
    if(low[to]>tin[v]) bridges.emplace_back(min(v,to),max(v,to)
        );
    if(p!=-1&&low[to]>=tin[v]) isArtPoint[v]=1;
    }
  }
  if(p==-1&&children>1) isArtPoint[v]=1;
}
};
```

EulerCycle.h

```
vector<ar<ll, 2>> gr[N];
vector<bool> vis_edge(N);
vll ans;
void dfs(ll node) {
    while(!gr[node].empty()) {
        auto [ch, ed] = gr[node].back(); gr[node].pop_back();
        if(vis_edge[ed]) continue;
        vis_edge[ed] = 1;
        dfs(ch);
    }
    ans.push_back(node);
}
main:
L(i, 1, m) {
    cin >> u >> v;
    gr[u].push_back({v, i}); gr[v].push_back({u, i});
    deg[u]++, deg[v]++;
}
L(i, 1, n) if(deg[i]&1) {cout << "IMPOSSIBLE" << nl; return;}
dfs(1);
if(sz(ans)!=m+1) {cout << "IMPOSSIBLE" << nl; return;}
for(ll x : ans) cout << x << gp;
```

6.5 Coloring

EdgeColoring.h

Description: Given a simple, undirected graph with max degree D, computes a $(D + 1)$-coloring of the edges such that no neighboring edges share a color. (D-coloring is NP-hard, but can be done for bipartite graphs by repeated matchings of max-degree nodes.)
Time: $\mathcal{O}(NM)$

```
vi edgeColoring(int N, vector<pii> eds) {
  vi cc(N + 1), ret(sz(eds)), fan(N), free(N), loc;
  for (pii e : eds) ++cc[e.first], ++cc[e.second];
  int u, v, ncols = *max_element(all(cc)) + 1;
  vector<vi> adj(N, vi(ncols, -1));
  for (pii e : eds) {
    tie(u, v) = e;
    fan[0] = v;
    loc.assign(ncols, 0);
    int at = u, end = u, d, c = free[u], ind = 0, i = 0;
    while (d = free[v], !loc[d] && (v = adj[u][d]) != -1)
      loc[d] = ++ind, cc[ind] = d, fan[ind] = v;
    cc[loc[d]] = c;
    for (int cd = d; at != -1; cd ^= c ^ d, at = adj[at][cd])
      swap(adj[at][cd], adj[end = at][cd ^ c ^ d]);
    while (adj[fan[i]][d] != -1) {
      int left = fan[i], right = fan[++i], e = cc[i];
      adj[u][e] = left;
      adj[left][e] = u;
      adj[right][e] = -1;
      free[right] = e;
    }
    adj[u][d] = fan[i];
    adj[fan[i]][d] = u;
    for (int y : {fan[0], u, end})
      for (int& z = free[y] = 0; adj[y][z] != -1; z++);
  }
  rep(i,0,sz(eds))
    for (tie(u, v) = eds[i]; adj[u][ret[i]] != v;) ++ret[i];
  return ret;
}
```

6.6 Trees

BinaryLifting.h

Description: Calculate power of two jumps in a tree, to support fast upward jumps and LCAs. Assumes the root node points to itself.
Time: construction $\mathcal{O}(N \log N)$, queries $\mathcal{O}(\log N)$

```
vector<vi> treeJump(vi& P){
  int on = 1, d = 1;
```

```
  while(on < sz(P)) on *= 2, d++;
  vector<vi> jmp(d, P);
  rep(i,1,d) rep(j,0,sz(P))
    jmp[i][j] = jmp[i-1][jmp[i-1][j]];
  return jmp;
}
int jmp(vector<vi>& tbl, int nod, int steps){
  rep(i,0,sz(tbl))
    if(steps&(1<<i)) nod = tbl[i][nod];
  return nod;
}
int lca(vector<vi>& tbl, vi& depth, int a, int b) {
  if (depth[a] < depth[b]) swap(a, b);
  a = jmp(tbl, a, depth[a] - depth[b]);
  if (a == b) return a;
  for (int i = sz(tbl); i--;) {
    int c = tbl[i][a], d = tbl[i][b];
    if (c != d) a = c, b = d;
  }
  return tbl[0][a];
}
```

LCA.h

Description: Data structure for computing lowest common ancestors in a tree (with 0 as root). C should be an adjacency list of the tree, either directed or undirected.
Time: $\mathcal{O}(N \log N + Q)$

"../data-structures/RMQ.h"

```
struct LCA {
  int T = 0;
  vi time, path, ret;
  RMQ<int> rmq;

  LCA(vector<vi>& C) : time(sz(C)), rmq((dfs(C,0,-1), ret)) {}
  void dfs(vector<vi>& C, int v, int par) {
    time[v] = T++;
    for (int y : C[v]) if (y != par) {
      path.push_back(v), ret.push_back(time[v]);
      dfs(C, y, v);
    }
  }

  int lca(int a, int b) {
    if (a == b) return a;
    tie(a, b) = minmax(time[a], time[b]);
    return path[rmq.query(a, b)];
  }
  //dist(a,b){return depth[a] + depth[b] - 2*depth[lca(a,b)];}
};
```

VirtualTree.h

Description: Given a rooted tree and a subset S of nodes, compute the minimal subtree that contains all the nodes by adding all (at most $|S| - 1$) pairwise LCA's and compressing edges. Returns a list of (par, orig_index) representing a tree rooted at 0. The root points to itself.
Time: $\mathcal{O}(|S| \log |S|)$

"LCA.h"

```
typedef vector<pair<int, int>> vpi;
vpi virtualTree(LCA& lca, const vi& subset) {
  static vi rev; rev.resize(sz(lca.time));
  vi li = subset, &T = lca.time;
  auto cmp = [&](int a, int b) { return T[a] < T[b]; };
  sort(all(li), cmp);
  int m = sz(li)-1;
  rep(i,0,m) {
    int a = li[i], b = li[i+1];
    li.push_back(lca.lca(a, b));
  }
  sort(all(li), cmp);
  li.erase(unique(all(li)), li.end());
  rep(i,0,sz(li)) rev[li[i]] = i;
  vpi ret = {pii(0, li[0])};
  rep(i,0,sz(li)-1) {
    int a = li[i], b = li[i+1];
    ret.emplace_back(rev[lca.lca(a, b)], b);
  }
  return ret;
}
```

HLD.h

Description: Decomposes a tree into vertex disjoint heavy paths and light edges such that the path from any leaf to the root contains at most log(n) light edges. Code does additive modifications and max queries, but can support commutative segtree modifications/queries on paths and subtrees. Takes as input the full adjacency list. VALS_EDGES being true means that values are stored in the edges, as opposed to the nodes. All values initialized to the segtree default. Root must be 0.
Time: $\mathcal{O}\left((\log N)^2\right)$

"../data-structures/LazySegmentTree.h"

```
template <bool VALS_EDGES> struct HLD {
  int N, tim = 0;
  vector<vi> adj;
  vi par, siz, rt, pos;
  Lseg *tree;
  HLD(vector<vi> adj_)
    : N(sz(adj_)), adj(adj_), par(N, -1), siz(N, 1),
      rt(N),pos(N),tree(new Lseg(0, N)){ dfsSz(0); dfsHld(0); }
  void dfsSz(int v) {
    for (int& u : adj[v]) {
      adj[u].erase(find(all(adj[u]), v));
      par[u] = v;
      dfsSz(u);
      siz[v] += siz[u];
      if (siz[u] > siz[adj[v][0]]) swap(u, adj[v][0]);
    }
  }
  void dfsHld(int v) {
    pos[v] = tim++;
    for (int u : adj[v]) {
      rt[u] = (u == adj[v][0] ? rt[v] : u);
      dfsHld(u);
    }
  }
  template <class B> void process(int u, int v, B op) {
    for (;; v = par[rt[v]]) {
      if (pos[u] > pos[v]) swap(u, v);
      if (rt[u] == rt[v]) break;
      op(pos[rt[v]], pos[v] + 1);
    }
    op(pos[u] + VALS_EDGES, pos[v] + 1);
  }
  void modifyPath(int u, int v, int val) {
    process(u, v, [&](int l, int r) { tree->add(l, r, val); });
  }
  void modifySubtree(int v, ll val) {
      tree->add(pos[v] + VALS_EDGES, pos[v] + siz[v], val);
  }
  int queryPath(int u, int v) { // Modify depending on problem
    int res = -1e9;
    process(u, v, [&](int l, int r) {
        res = max(res, (tree->query(l, r)).mx );
    });
    return res;
  }
  int querySubtree(int v) { // modifySubtree is similar
    return (tree->query(pos[v] + VALS_EDGES, pos[v] + siz[v])).
        mx;
  }
};
```

LinkCutTree.h

Description: Represents a forest of unrooted trees. You can add and remove edges (as long as the result is still a forest), and check whether two nodes are in the same tree.
Time: All operations take amortized $\mathcal{O}(\log N)$.

```
struct Node { // Splay tree. Root's pp contains tree's parent.
  Node *p = 0, *pp = 0, *c[2];
  bool flip = 0;
  Node() { c[0] = c[1] = 0; fix(); }
  void fix() {
    if (c[0]) c[0]->p = this;
    if (c[1]) c[1]->p = this;
    // (+ update sum of subtree elements etc. if wanted)
  }
  void pushFlip() {
    if (!flip) return;
    flip = 0; swap(c[0], c[1]);
    if (c[0]) c[0]->flip ^= 1;
    if (c[1]) c[1]->flip ^= 1;
  }
  int up() { return p ? p->c[1] == this : -1; }
  void rot(int i, int b) {
    int h = i ^ b;
    Node *x = c[i], *y = b == 2 ? x : x->c[h], *z = b ? y : x;
    if ((y->p = p)) p->c[up()] = y;
    c[i] = z->c[i ^ 1];
    if (b < 2) {
      x->c[h] = y->c[h ^ 1];
      y->c[h ^ 1] = x;
    }
    z->c[i ^ 1] = this;
    fix(); x->fix(); y->fix();
    if (p) p->fix();
    swap(pp, y->pp);
  }
  void splay() {
    for (pushFlip(); p; ) {
      if (p->p) p->p->pushFlip();
      p->pushFlip(); pushFlip();
      int c1 = up(), c2 = p->up();
      if (c2 == -1) p->rot(c1, 2);
      else p->p->rot(c2, c1 != c2);
    }
  }
  Node* first() {
    pushFlip();
    return c[0] ? c[0]->first() : (splay(), this);
  }
};

struct LinkCut {
  vector<Node> node;
  LinkCut(int N) : node(N) {}

  void link(int u, int v) { // add an edge (u, v)
    assert(!connected(u, v));
    makeRoot(&node[u]);
    node[u].pp = &node[v];
  }
  void cut(int u, int v) { // remove an edge (u, v)
    Node *x = &node[u], *top = &node[v];
    makeRoot(top); x->splay();
    assert(top == (x->pp ?: x->c[0]));
```

```
    if (x->pp) x->pp = 0;
    else {
      x->c[0] = top->p = 0;
      x->fix();
    }
  }
  bool connected(int u, int v) { // are u, v in the same tree?
    Node* nu = access(&node[u])->first();
    return nu == access(&node[v])->first();
  }
  void makeRoot(Node* u) {
    access(u);
    u->splay();
    if(u->c[0]) {
      u->c[0]->p = 0;
      u->c[0]->flip ^= 1;
      u->c[0]->pp = u;
      u->c[0] = 0;
      u->fix();
    }
  }
  Node* access(Node* u) {
    u->splay();
    while (Node* pp = u->pp) {
      pp->splay(); u->pp = 0;
      if (pp->c[1]) {
        pp->c[1]->p = 0; pp->c[1]->pp = pp; }
      pp->c[1] = u; pp->fix(); u = pp;
    }
    return u;
  }
};
```

DirectedMST.h

Description: Finds a minimum spanning tree/arborescence of a directed graph, given a root node. If no MST exists, returns -1.
Time: $\mathcal{O}(E \log V)$

"../data-structures/UnionFindRollback.h"

```
struct Edge { int a, b; ll w; };
struct Node {
  Edge key;
  Node *l, *r;
  ll delta;
  void prop() {
    key.w += delta;
    if (l) l->delta += delta;
    if (r) r->delta += delta;
    delta = 0;
  }
  Edge top() { prop(); return key; }
};
Node *merge(Node *a, Node *b) {
  if (!a || !b) return a ?: b;
  a->prop(), b->prop();
  if (a->key.w > b->key.w) swap(a, b);
  swap(a->l, (a->r = merge(b, a->r)));
  return a;
}
void pop(Node*& a) { a->prop(); a = merge(a->l, a->r); }

pair<ll, vi> dmst(int n, int r, vector<Edge>& g) {
  RollbackUF uf(n);
  vector<Node*> heap(n);
  for (Edge e : g) heap[e.b] = merge(heap[e.b], new Node{e});
  ll res = 0;
  vi seen(n, -1), path(n), par(n);
  seen[r] = r;
  vector<Edge> Q(n), in(n, {-1,-1}), comp;
  deque<tuple<int, int, vector<Edge>>> cycs;
  rep(s,0,n) {
    int u = s, qi = 0, w;
    while (seen[u] < 0) {
      if (!heap[u]) return {-1,{}};
      Edge e = heap[u]->top();
      heap[u]->delta -= e.w, pop(heap[u]);
      Q[qi] = e, path[qi++] = u, seen[u] = s;
      res += e.w, u = uf.find(e.a);
      if (seen[u] == s) {
        Node* cyc = 0;
        int end = qi, time = uf.time();
        do cyc = merge(cyc, heap[w = path[--qi]]);
        while (uf.join(u, w));
        u = uf.find(u), heap[u] = cyc, seen[u] = -1;
        cycs.push_front({u, time, {&Q[qi], &Q[end]}});
      }
    }
    rep(i,0,qi) in[uf.find(Q[i].b)] = Q[i];
  }

  for (auto& [u,t,comp] : cycs) { // restore sol (optional)
    uf.rollback(t);
    Edge inEdge = in[u];
    for (auto& e : comp) in[uf.find(e.b)] = e;
    in[uf.find(inEdge.b)] = inEdge;
  }
  rep(i,0,n) par[i] = in[i].a;
  return {res, par};}
```

TreeFlatten.h

Description: Subtree of a node is [in[node]...out[node]] The node itself is [in[node]...in[node]]

```
vll in(N), out(N);   ll timer = -1;
void euler_tour(ll node, ll par) {
    in[node] = ++timer;
    for(ll ch : gr[node]) {
        if(ch == par) continue;
        euler_tour(ch, node);
    }
    out[node] = timer;
}
//  usage :
    euler_tour(1, 0);
    vll flat_tree(n);
    L(i, 1, n) flat_tree[in[i]] = a[i];
    sumSeg seg(flat_tree);
```

CentDecom.h

```
ll n,m,x,y,z,q,k,u,v,w;
vector<int> gr[N];
int tot, sz[N], done[N], cenpar[N];

void calc_sz(int node,int p){
 tot++; sz[node]=1;
 for(auto ch:gr[node]){
  if(ch==p||done[ch]) continue;
  calc_sz(ch,node); sz[node]+=sz[ch];
 }
}

int find_cen(int node,int p){ // find centroid
 for(auto ch:gr[node]){
  if(ch==p||done[ch]) continue;
  else if(sz[ch]>tot/2) return find_cen(ch,node);
 }
 return node;
}

void decompose(int node,int p){ // find centroid of subtree
 tot=0; calc_sz(node,p);
 int cen=find_cen(node,p);
 cenpar[cen]=p; done[cen]=1;
 for(auto ch:gr[cen]){
  if(ch==p||done[ch]) continue;
  decompose(ch,cen);
 }
}

vll cen_tree[N]; // centroid tree

int form_cen_tree(){ // form graph, return root(centroid)
 decompose(1,0); int root;
 L(i,1,n){
  if(cenpar[i]==0) root=i;
  if(cenpar[i]!=0){
    cen_tree[i].push_back(cenpar[i]);
    cen_tree[cenpar[i]].push_back(i);
   }
 }
 return root;
}
```

TreePathsIntersection.h

Description: Find intersecting path between two paths a..b and c..d

```
// returns {-1,-1} if empty
pair<int,int> intersect(int a, int b, int c, int d) {
    int cand[4] = { lca(a,c), lca(a,d), lca(b,c), lca(b,d) };
    sort(cand, cand+4, [&](int x, int y) {
        return dep[x]>dep[y];
    } );
    int u = cand[0], v = cand[1];
    if (dep[u] < max(dep[lca(a,b)], dep[lca(c,d)]))
        return {-1, -1};
    return {u, v};
}
```

6.7 Math

6.7.1 Number of Spanning Trees

Create an $N \times N$ matrix `mat`, and for each edge $a \to b \in G$, do `mat[a][b]--, mat[b][b]++` (and `mat[b][a]--, mat[a][a]++` if G is undirected). Remove the ith row and column and take the determinant; this yields the number of directed spanning trees rooted at i. For undirected G, remove any row or column.

6.7.2 Proper Cycle Colorings

The number of proper colorings of the vertices of a cycle C_n using k colors is

$$P_{C_n}(k) = (k-1)^n + (-1)^n(k-1).$$

6.7.3 Erdős–Gallai theorem

A simple graph with node degrees $d_1 \geq \cdots \geq d_n$ exists iff $d_1 + \cdots + d_n$ is even and for every $k = 1 \ldots n$,

$$\sum_{i=1}^{k} d_i \leq k(k-1) + \sum_{i=k+1}^{n} \min(d_i, k).$$

Geometry (7)

7.1 Geometric primitives

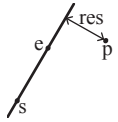
Point.h

Description: Class to handle points in the plane. T can be e.g. double or long long. (Avoid int.)

```
template <class T> int sgn(T x) { return (x > 0) - (x < 0); }
template<class T>
struct Point {
  typedef Point P;
  T x, y;
  explicit Point(T x=0, T y=0) : x(x), y(y) {}
  bool operator<(P p) const { return tie(x,y) < tie(p.x,p.y); }
  bool operator==(P p) const { return tie(x,y)==tie(p.x,p.y); }
  P operator+(P p) const { return P(x+p.x, y+p.y); }
  P operator-(P p) const { return P(x-p.x, y-p.y); }
  P operator*(T d) const { return P(x*d, y*d); }
  P operator/(T d) const { return P(x/d, y/d); }
  T dot(P p) const { return x*p.x + y*p.y; }
  T cross(P p) const { return x*p.y - y*p.x; }
  T cross(P a, P b) const { return (a-*this).cross(b-*this); }
  T dist2() const { return x*x + y*y; }
  double dist() const { return sqrt((double)dist2()); }
  // angle to x-axis in interval [-pi, pi]
  double angle() const { return atan2(y, x); }
  P unit() const { return *this/dist(); } // makes dist()=1
  P perp() const { return P(-y, x); } // rotates +90 degrees
  P normal() const { return perp().unit(); }
  // returns point rotated 'a' radians ccw around the origin
  P rotate(double a) const {
    return P(x*cos(a)-y*sin(a),x*sin(a)+y*cos(a)); }
  friend ostream& operator<<(ostream& os, P p) {
    return os << "(" << p.x << "," << p.y << ")"; }
};
```

lineDistance.h

Description:
Returns the signed distance between point p and the line containing points a and b. Positive value on left side and negative on right as seen from a towards b. a==b gives nan. P is supposed to be Point<T> or Point3D<T> where T is e.g. double or long long. It uses products in intermediate steps so watch out for overflow if using int or long long. Using Point3D will always give a non-negative distance. For Point3D, call .dist on the result of the cross product.

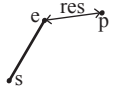


"Point.h"

```
template<class P>
double lineDist(const P& a, const P& b, const P& p) {
  return (double)(b-a).cross(p-a)/(b-a).dist();
}
```

SegmentDistance.h

Description:
Returns the shortest distance between point p and the line segment from point s to e.



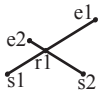
Usage:
```
Point<double> a, b(2,2), p(1,1);
bool onSegment = segDist(a,b,p) < 1e-10;
```

"Point.h"

```
typedef Point<double> P;
double segDist(P& s, P& e, P& p) {
  if (s==e) return (p-s).dist();
  auto d = (e-s).dist2(), t = min(d,max(.0,(p-s).dot(e-s)));
  return ((p-s)*d-(e-s)*t).dist()/d;
}
```

SegmentIntersection.h

Description:
If a unique intersection point between the line segments going from s1 to e1 and from s2 to e2 exists then it is returned. If no intersection point exists an empty vector is returned. If infinitely many exist a vector with 2 elements is returned, containing the endpoints of the common line segment. The wrong position will be returned if P is Point<ll> and the intersection point does not have integer coordinates. Products of three coordinates are used in intermediate steps so watch out for overflow if using int or long long.



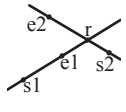
Usage:
```
vector<P> inter = segInter(s1,e1,s2,e2);
if (sz(inter)==1)
cout << "segments intersect at " << inter[0] << endl;
```

"Point.h", "OnSegment.h"

```
template<class P> vector<P> segInter(P a, P b, P c, P d) {
  auto oa = c.cross(d, a), ob = c.cross(d, b),
       oc = a.cross(b, c), od = a.cross(b, d);
  // Checks if intersection is single non-endpoint point.
  if (sgn(oa) * sgn(ob) < 0 && sgn(oc) * sgn(od) < 0)
    return {(a * ob - b * oa) / (ob - oa)};
  set<P> s;
  if (onSegment(c, d, a)) s.insert(a);
  if (onSegment(c, d, b)) s.insert(b);
  if (onSegment(a, b, c)) s.insert(c);
  if (onSegment(a, b, d)) s.insert(d);
  return {all(s)};
}
```

lineIntersection.h

Description:
If a unique intersection point of the lines going through s1,e1 and s2,e2 exists {1, point} is returned. If no intersection point exists {0, (0,0)} is returned and if infinitely many exists {-1, (0,0)} is returned. The wrong position will be returned if P is Point<ll> and the intersection point does not have integer coordinates. Products of three coordinates are used in intermediate steps so watch out for overflow if using int or ll.



Usage:
```
auto res = lineInter(s1,e1,s2,e2);
if (res.first == 1)
cout << "intersection point at " << res.second << endl;
```

"Point.h"

```
template<class P>
pair<int, P> lineInter(P s1, P e1, P s2, P e2) {
  auto d = (e1 - s1).cross(e2 - s2);
  if (d == 0) // if parallel
    return {-(s1.cross(e1, s2) == 0), P(0, 0)};
  auto p = s2.cross(e1, e2), q = s2.cross(e2, s1);
  return {1, (s1 * p + e1 * q) / d};
}
```

sideOf.h

Description: Returns where p is as seen from s towards e. 1/0/-1 $\Leftrightarrow$ left/on line/right. If the optional argument eps is given 0 is returned if p is within distance eps from the line. P is supposed to be Point<T> where T is e.g. double or long long. It uses products in intermediate steps so watch out for overflow if using int or long long.

Usage:
```
bool left = sideOf(p1,p2,q)==1;
```

"Point.h"

```
template<class P>
int sideOf(P s, P e, P p) { return sgn(s.cross(e, p)); }

template<class P>
int sideOf(const P& s, const P& e, const P& p, double eps) {
  auto a = (e-s).cross(p-s);
  double l = (e-s).dist()*eps;
  return (a > l) - (a < -l);
}
```

OnSegment.h

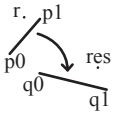
Description: Returns true iff p lies on the line segment from s to e. Use (segDist(s,e,p)<=epsilon) instead when using Point<double>.

"Point.h"

```
template<class P> bool onSegment(P s, P e, P p) {
  return p.cross(s, e) == 0 && (s - p).dot(e - p) <= 0;
}
```

linearTransformation.h

Description:
Apply the linear transformation (translation, rotation and scaling) which takes line p0-p1 to line q0-q1 to point r.



"Point.h"

```
typedef Point<double> P;
P linearTransformation(const P& p0, const P& p1,
    const P& q0, const P& q1, const P& r) {
  P dp = p1-p0, dq = q1-q0, num(dp.cross(dq), dp.dot(dq));
  return q0 + P((r-p0).cross(num), (r-p0).dot(num))/dp.dist2();
}
```

LineProjectionReflection.h

Description: Projects point p onto line ab. Set refl=true to get reflection of point p across line ab instead. The wrong point will be returned if P is an integer point and the desired point doesn't have integer coordinates. Products of three coordinates are used in intermediate steps so watch out for overflow.

"Point.h"

```
template<class P>
P lineProj(P a, P b, P p, bool refl=false) {
  P v = b - a;
  return p - v.perp()*(1+refl)*v.cross(p-a)/v.dist2();
}
```

Angle.h

Description: A class for ordering angles (as represented by int points and a number of rotations around the origin). Useful for rotational sweeping. Sometimes also represents points or vectors.

Usage:
```
vector<Angle> v = {w[0], w[0].t360() ...}; // sorted
int j = 0; rep(i,0,n) { while (v[j] < v[i].t180()) ++j; }
// sweeps j such that (j-i) represents the number of positively
oriented triangles with vertices at 0 and i
```

```
struct Angle {
  int x, y;
  int t;
  Angle(int x, int y, int t=0) : x(x), y(y), t(t) {}
  Angle operator-(Angle b) const { return {x-b.x, y-b.y, t}; }
  int half() const {
    assert(x || y);
    return y < 0 || (y == 0 && x < 0);
  }
  Angle t90() const { return {-y, x, t + (half() && x >= 0)}; }
  Angle t180() const { return {-x, -y, t + half()}; }
  Angle t360() const { return {x, y, t + 1}; }
};
bool operator<(Angle a, Angle b) {
  // add a.dist2() and b.dist2() to also compare distances
  return make_tuple(a.t, a.half(), a.y * (ll)b.x) <
         make_tuple(b.t, b.half(), a.x * (ll)b.y);
}

// Given two points, this calculates the smallest angle between
// them, i.e., the angle that covers the defined line segment.
pair<Angle, Angle> segmentAngles(Angle a, Angle b) {
  if (b < a) swap(a, b);
  return (b < a.t180() ?
          make_pair(a, b) : make_pair(b, a.t360()));
}
```

```
Angle operator+(Angle a, Angle b) { // point a + vector b
  Angle r(a.x + b.x, a.y + b.y, a.t);
  if (a.t180() < r) r.t--;
  return r.t180() < a ? r.t360() : r;
}
Angle angleDiff(Angle a, Angle b) { // angle b - angle a
  int tu = b.t - a.t; a.t = b.t;
  return {a.x*b.x + a.y*b.y, a.x*b.y - a.y*b.x, tu - (b < a)};
}
```

7.2 Circles

CircleIntersection.h

Description: Computes the pair of points at which two circles intersect. Returns false in case of no intersection.

"Point.h"

```
typedef Point<double> P;
bool circleInter(P a,P b,double r1,double r2,pair<P, P>* out) {
  if (a == b) { assert(r1 != r2); return false; }
  P vec = b - a;
  double d2 = vec.dist2(), sum = r1+r2, dif = r1-r2,
         p = (d2 + r1*r1 - r2*r2)/(d2*2), h2 = r1*r1 - p*p*d2;
  if (sum*sum < d2 || dif*dif > d2) return false;
  P mid = a + vec*p, per = vec.perp() * sqrt(fmax(0, h2) / d2);
  *out = {mid + per, mid - per};
  return true;
}
```

CircleTangents.h

Description: Finds the external tangents of two circles, or internal if r2 is negated. Can return 0, 1, or 2 tangents – 0 if one circle contains the other (or overlaps it, in the internal case, or if the circles are the same); 1 if the circles are tangent to each other (in which case .first = .second and the tangent line is perpendicular to the line between the centers). .first and .second give the tangency points at circle 1 and 2 respectively. To find the tangents of a circle with a point set r2 to 0.

"Point.h"

```
template<class P>
vector<pair<P, P>> tangents(P c1, double r1, P c2, double r2) {
  P d = c2 - c1;
  double dr = r1 - r2, d2 = d.dist2(), h2 = d2 - dr * dr;
  if (d2 == 0 || h2 < 0)  return {};
  vector<pair<P, P>> out;
  for (double sign : {-1, 1}) {
    P v = (d * dr + d.perp() * sqrt(h2) * sign) / d2;
    out.push_back({c1 + v * r1, c2 + v * r2});
  }
  if (h2 == 0) out.pop_back();
  return out;
}
```

CircleLine.h

Description: Finds the intersection between a circle and a line. Returns a vector of either 0, 1, or 2 intersection points. P is intended to be Point<double>.

"Point.h"

```
template<class P>
vector<P> circleLine(P c, double r, P a, P b) {
  P ab = b - a, p = a + ab * (c-a).dot(ab) / ab.dist2();
  double s = a.cross(b, c), h2 = r*r - s*s / ab.dist2();
  if (h2 < 0) return {};
  if (h2 == 0) return {p};
  P h = ab.unit() * sqrt(h2);
  return {p - h, p + h};
}
```

CirclePolygonIntersection.h

Description: Returns the area of the intersection of a circle with a ccw polygon.
Time: $\mathcal{O}(n)$

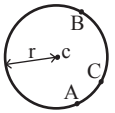
"../../content/geometry/Point.h"

```
typedef Point<double> P;
#define arg(p, q) atan2(p.cross(q), p.dot(q))
double circlePoly(P c, double r, vector<P> ps) {
  auto tri = [&](P p, P q) {
    auto r2 = r * r / 2;
    P d = q - p;
    auto a = d.dot(p)/d.dist2(), b = (p.dist2()-r*r)/d.dist2();
    auto det = a * a - b;
    if (det <= 0) return arg(p, q) * r2;
    auto s = max(0., -a-sqrt(det)), t = min(1., -a+sqrt(det));
    if (t < 0 || 1 <= s) return arg(p, q) * r2;
    P u = p + d * s, v = q + d * (t-1);
    return arg(p,u) * r2 + u.cross(v)/2 + arg(v,q) * r2;
  };
  auto sum = 0.0;
  rep(i,0,sz(ps))
    sum += tri(ps[i] - c, ps[(i + 1) % sz(ps)] - c);
  return sum;
}
```

circumcircle.h

Description:

The circumcirle of a triangle is the circle intersecting all three vertices. ccRadius returns the radius of the circle going through points A, B and C and ccCenter returns the center of the same circle.



"Point.h"

```
typedef Point<double> P;
double ccRadius(const P& A, const P& B, const P& C) {
  return (B-A).dist()*(C-B).dist()*(A-C).dist()/
      abs((B-A).cross(C-A))/2;
}
P ccCenter(const P& A, const P& B, const P& C) {
  P b = C-A, c = B-A;
  return A + (b*c.dist2()-c*b.dist2()).perp()/b.cross(c)/2;
}
```

MinimumEnclosingCircle.h

Description: Computes the minimum circle that encloses a set of points.
Time: expected $\mathcal{O}(n)$

"circumcircle.h"

```
pair<P, double> mec(vector<P> ps) {
  shuffle(all(ps), mt19937(time(0)));
  P o = ps[0];
  double r = 0, EPS = 1 + 1e-8;
  rep(i,0,sz(ps)) if ((o - ps[i]).dist() > r * EPS) {
    o = ps[i], r = 0;
    rep(j,0,i) if ((o - ps[j]).dist() > r * EPS) {
      o = (ps[i] + ps[j]) / 2;
      r = (o - ps[i]).dist();
      rep(k,0,j) if ((o - ps[k]).dist() > r * EPS) {
        o = ccCenter(ps[i], ps[j], ps[k]);
        r = (o - ps[i]).dist();
      }
    }
  }
  return {o, r};
}
```

7.3 Polygons

InsidePolygon.h

Description: Returns true if p lies within the polygon. If strict is true, it returns false for points on the boundary. The algorithm uses products in intermediate steps so watch out for overflow.
Usage: vector<P> v = {P{4,4}, P{1,2}, P{2,1}};
bool in = inPolygon(v, P{3, 3}, false);
Time: $\mathcal{O}(n)$

"Point.h", "OnSegment.h", "SegmentDistance.h"

```
template<class P>
bool inPolygon(vector<P> &p, P a, bool strict = true) {
  int cnt = 0, n = sz(p);
  rep(i,0,n) {
    P q = p[(i + 1) % n];
    if (onSegment(p[i], q, a)) return !strict;
    //or: if (segDist(p[i], q, a) <= eps) return !strict;
    cnt ^= ((a.y<p[i].y) - (a.y<q.y)) * a.cross(p[i], q) > 0;
  }
  return cnt;
}
```

PolygonArea.h

Description: Returns twice the signed area of a polygon. Clockwise enumeration gives negative area. Watch out for overflow if using int as T!

"Point.h"

```
template<class T>
T polygonArea2(vector<Point<T>>& v) {
  T a = v.back().cross(v[0]);
  rep(i,0,sz(v)-1) a += v[i].cross(v[i+1]);
  return a;
}
```

PolygonCenter.h

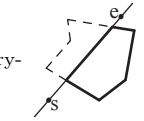
Description: Returns the center of mass for a polygon.
Time: $\mathcal{O}(n)$

"Point.h"

```
typedef Point<double> P;
P polygonCenter(const vector<P>& v) {
  P res(0, 0); double A = 0;
  for (int i = 0, j = sz(v) - 1; i < sz(v); j = i++) {
    res = res + (v[i] + v[j]) * v[j].cross(v[i]);
    A += v[j].cross(v[i]);
  }
  return res / A / 3;
}
```

PolygonCut.h

Description:
Returns a vector with the vertices of a polygon with everything to the left of the line going from s to e cut away.
Usage: vector<P> p = ...;
p = polygonCut(p, P(0,0), P(1,0));



"Point.h"

```
typedef Point<double> P;
vector<P> polygonCut(const vector<P>& poly, P s, P e) {
  vector<P> res;
  rep(i,0,sz(poly)) {
    P cur = poly[i], prev = i ? poly[i-1] : poly.back();
    auto a = s.cross(e, cur), b = s.cross(e, prev);
    if ((a < 0) != (b < 0))
      res.push_back(cur + (prev - cur) * (a / (a - b)));
    if (a < 0)
      res.push_back(cur);
  }
  return res;
}
```

PolygonUnion.h

Description: Calculates the area of the union of n polygons (not necessarily convex). The points within each polygon must be given in CCW order. (Epsilon checks may optionally be added to sideOf/sgn, but shouldn't be needed.)
Time: $\mathcal{O}\left(N^2\right)$, where N is the total number of points
"Point.h", "sideOf.h"

```
typedef Point<double> P;
double rat(P a, P b) { return sgn(b.x) ? a.x/b.x : a.y/b.y; }
double polyUnion(vector<vector<P>>& poly) {
  double ret = 0;
  rep(i,0,sz(poly)) rep(v,0,sz(poly[i])) {
    P A = poly[i][v], B = poly[i][(v + 1) % sz(poly[i])];
    vector<pair<double, int>> segs = {{0, 0}, {1, 0}};
    rep(j,0,sz(poly)) if (i != j) {
      rep(u,0,sz(poly[j])) {
        P C = poly[j][u], D = poly[j][(u + 1) % sz(poly[j])];
        int sc = sideOf(A, B, C), sd = sideOf(A, B, D);
        if (sc != sd) {
          double sa = C.cross(D, A), sb = C.cross(D, B);
          if (min(sc, sd) < 0)
            segs.emplace_back(sa / (sa - sb), sgn(sc - sd));
        } else if (!sc && !sd && j<i && sgn((B-A).dot(D-C))>0){
          segs.emplace_back(rat(C - A, B - A), 1);
          segs.emplace_back(rat(D - A, B - A), -1);
        }
      }
    }
    sort(all(segs));
    for (auto& s : segs) s.first = min(max(s.first, 0.0), 1.0);
    double sum = 0;
    int cnt = segs[0].second;
    rep(j,1,sz(segs)) {
      if (!cnt) sum += segs[j].first - segs[j - 1].first;
      cnt += segs[j].second;
    }
    ret += A.cross(B) * sum;
  }
  return ret / 2;
}
```

ConvexHull.h

Description:
Returns a vector of the points of the convex hull in counterclockwise order. Points on the edge of the hull between two other points are not considered part of the hull.



Time: $\mathcal{O}\left(n \log n\right)$
"Point.h"

```
typedef Point<ll> P;
vector<P> convexHull(vector<P> pts) {
  if (sz(pts) <= 1) return pts;
  sort(all(pts));
  vector<P> h(sz(pts)+1);
  int s = 0, t = 0;
  for (int it = 2; it--; s = --t, reverse(all(pts)))
    for (P p : pts) {
      while (t >= s + 2 && h[t-2].cross(h[t-1], p) <= 0) t--;
      h[t++] = p;
    }
  return {h.begin(), h.begin() + t - (t == 2 && h[0] == h[1])};
}
```

HullDiameter.h

Description: Returns the two points with max distance on a convex hull (ccw, no duplicate/collinear points).
Time: $\mathcal{O}\left(n\right)$
"Point.h"

```
typedef Point<ll> P;
array<P, 2> hullDiameter(vector<P> S) {
  int n = sz(S), j = n < 2 ? 0 : 1;
  pair<ll, array<P, 2>> res({0, {S[0], S[0]}});
  rep(i,0,j)
    for (;; j = (j + 1) % n) {
      res = max(res, {(S[i] - S[j]).dist2(), {S[i], S[j]}});
      if ((S[(j + 1) % n] - S[j]).cross(S[i + 1] - S[i]) >= 0)
        break;
    }
  return res.second;
}
```

PointInsideHull.h

Description: Determine whether a point t lies inside a convex hull (CCW order, with no collinear points). Returns true if point lies within the hull. If strict is true, points on the boundary aren't included.
Time: $\mathcal{O}\left(\log N\right)$
"Point.h", "sideOf.h", "OnSegment.h"

```
typedef Point<ll> P;

bool inHull(const vector<P>& l, P p, bool strict = true) {
  int a = 1, b = sz(l) - 1, r = !strict;
  if (sz(l) < 3) return r && onSegment(l[0], l.back(), p);
  if (sideOf(l[0], l[a], l[b]) > 0) swap(a, b);
  if (sideOf(l[0], l[a], p) >= r || sideOf(l[0], l[b], p)<= -r)
    return false;
  while (abs(a - b) > 1) {
    int c = (a + b) / 2;
    (sideOf(l[0], l[c], p) > 0 ? b : a) = c;
  }
  return sgn(l[a].cross(l[b], p)) < r;
}
```

LineHullIntersection.h

Description: Line-convex polygon intersection. The polygon must be ccw and have no collinear points. lineHull(line, poly) returns a pair describing the intersection of a line with the polygon: $\bullet$ $(-1,-1)$ if no collision, $\bullet$ $(i,-1)$ if touching the corner i, $\bullet$ (i,i) if along side $(i,i+1)$, $\bullet$ (i,j) if crossing sides $(i,i+1)$ and $(j,j+1)$. In the last case, if a corner i is crossed, this is treated as happening on side $(i,i+1)$. The points are returned in the same order as the line hits the polygon. `extrVertex` returns the point of a hull with the max projection onto a line.
Time: $\mathcal{O}\left(\log n\right)$
"Point.h"

```
#define cmp(i,j) sgn(dir.perp().cross(poly[(i)%n]-poly[(j)%n]))
#define extr(i) cmp(i + 1, i) >= 0 && cmp(i, i - 1 + n) < 0
template <class P> int extrVertex(vector<P>& poly, P dir) {
  int n = sz(poly), lo = 0, hi = n;
  if (extr(0)) return 0;
  while (lo + 1 < hi) {
    int m = (lo + hi) / 2;
    if (extr(m)) return m;
    int ls = cmp(lo + 1, lo), ms = cmp(m + 1, m);
    (ls < ms || (ls == ms && ls == cmp(lo, m)) ? hi : lo) = m;
  }
  return lo;
}

#define cmpL(i) sgn(a.cross(poly[i], b))
template <class P>
array<int, 2> lineHull(P a, P b, vector<P>& poly) {
  int endA = extrVertex(poly, (a - b).perp());
  int endB = extrVertex(poly, (b - a).perp());
  if (cmpL(endA) < 0 || cmpL(endB) > 0)
    return {-1, -1};
  array<int, 2> res;
  rep(i,0,2) {
    int lo = endB, hi = endA, n = sz(poly);
    while ((lo + 1) % n != hi) {
      int m = ((lo + hi + (lo < hi ? 0 : n)) / 2) % n;
      (cmpL(m) == cmpL(endB) ? lo : hi) = m;
    }
    res[i] = (lo + !cmpL(hi)) % n;
    swap(endA, endB);
  }
  if (res[0] == res[1]) return {res[0], -1};
  if (!cmpL(res[0]) && !cmpL(res[1]))
    switch ((res[0] - res[1] + sz(poly) + 1) % sz(poly)) {
      case 0: return {res[0], res[0]};
      case 2: return {res[1], res[1]};
    }
  return res;
}
```

7.4 Misc. Point Set Problems

ClosestPair.h

Description: Finds the closest pair of points.
Time: $\mathcal{O}\left(n \log n\right)$
"Point.h"

```
typedef Point<ll> P;
pair<P, P> closest(vector<P> v) {
  assert(sz(v) > 1);
  set<P> S;
  sort(all(v), [](P a, P b) { return a.y < b.y; });
  pair<ll, pair<P, P>> ret{LLONG_MAX, {P(), P()}};
  int j = 0;
  for (P p : v) {
    P d{1 + (ll)sqrt(ret.first), 0};
    while (v[j].y <= p.y - d.x) S.erase(v[j++]);
    auto lo = S.lower_bound(p - d), hi = S.upper_bound(p + d);
    for (; lo != hi; ++lo)
      ret = min(ret, {(*lo - p).dist2(), {*lo, p}});
    S.insert(p);
  }
  return ret.second;
}
```

Strings (8)

KMP.h

```
// for each position (0 based***) of s, what is the best match
// of a suffix at that position with a prefix of s
vector<int> prefixFunction(const string& s){
  int n=s.size();  vector<int> pi(n);  pi[0]=0;
  for(int i=1;  i<n;  ++i){
    int j = pi[i-1];
    while(j>0 and s[j] != s[i]) j = pi[j-1];
    if(s[j] == s[i]) j++;
    pi[i]=j;
  }
  return pi;
}
// if the automaton is built on string s, then :
// aut[j][ch] = if i'm scanning/constructing some string and rn
// the best match between prefix_of_s and suffix_of_MY_string
// is of LENGTH j, and the next scanned/put character is ch,
// then what would be the best matched LENGTH
vector<vector<int>> prefixAutomaton(const string& s){
  int m = s.size();
  vector<int> pi = prefixFunction(s);
  const int alph = 26;
  vector<vector<int>> aut(m+1, vector<int>(alph));
  for(int j = 0;  j <= m;  j++){
    for(int c=0;  c<alph;  c++){
    if(j<m  &&  s[j] == char('a'+c)) aut[j][c]= j+1;
```

```
    else if(j == 0) aut[j][c] = 0;
    else aut[j][c] = aut[pi[j-1]][c];
    }
  }
 return aut;
}
```

Zfunc.h

Description: z[i] computes the length of the longest common prefix of s[i:] and s, except z[0] = 0. (abacaba -> 0010301)
Time: $\mathcal{O}(n)$

```
vi Z(const string& s) {
  vi z(sz(s));
  int l = -1, r = -1;
  rep(i,1,sz(s)) {
    z[i] = i >= r ? 0 : min(r - i, z[i - l]);
    while (i + z[i] < sz(s) && s[i + z[i]] == s[z[i]])
      z[i]++;
    if (i + z[i] > r)
      l = i, r = i + z[i];
  }
  return z;
}
```

Manacher.h

Description: For each position in a string, computes p[0][i] = radius of longest even palindrome where central pair is (i-1, i), p[1][i] = radius of longest odd (center EXCLUDED).
Time: $\mathcal{O}(N)$

```
array<vi, 2> manacher(const string& s) {
  int n = sz(s);
  array<vi,2> p = {vi(n+1), vi(n)};
  rep(z,0,2) for (int i=0, l=0, r=0;  i < n;  i++) {
    int t = r-i+!z;
    if (i<r) p[z][i] = min(t,  p[z][l+t]);
    int L = i - p[z][i],  R = i + p[z][i] - !z;
    while (L>=1 && R+1<n && s[L-1] == s[R+1])
      p[z][i]++, L--, R++;
    if (R>r) l=L, r=R;
  }
  return p;
}
```

Trie.h

```
struct Trie{
 static const int ALPHABET=26;
 static const char BASE='a';
 struct Node{
  int next[ALPHABET];
  int ends_here_cnt,count; // passes this node
  Node(){
   fill(next,next+ALPHABET,-1);
   ends_here_cnt=0; count=0;
  }
 };
 vector<Node> nodes;
 Trie(){ nodes.emplace_back(); } // root node
 void insert(const string& s){
  int cur=0;
  for(char ch:s){
   int c = ch-BASE;
   if(nodes[cur].next[c] == -1){
    nodes[cur].next[c]=nodes.size(); nodes.emplace_back();
   }
   cur = nodes[cur].next[c];  nodes[cur].count++;
  }
  nodes[cur].ends_here_cnt++;
 }
 bool erase(const string& s){
  if(!search(s)) return false;
  int cur=0;
  for(char ch:s){
   int c=ch-BASE;
   cur=nodes[cur].next[c];  nodes[cur].count--;
  }
  nodes[cur].ends_here_cnt--;
  return true;
 }
 bool search(const string& s)const{
  int cur=0;
  for(char ch:s){
   int c=ch-BASE;
   if(nodes[cur].next[c]==-1
      || nodes[nodes[cur].next[c]].count==0) return false;
   cur=nodes[cur].next[c];
  }
  return nodes[cur].ends_here_cnt>0;
}
 bool starts_with(const string& prefix)const{
  int cur=0;
  for(char ch:prefix){
   int c=ch-BASE;
   if(nodes[cur].next[c]==-1
     ||nodes[nodes[cur].next[c]].count==0) return false;
   cur=nodes[cur].next[c];
  }
  return true;
 }
 int count_prefix(const string& prefix)const{
  int cur=0;
  for(char ch:prefix){
   int c=ch-BASE;
   if(nodes[cur].next[c]==-1
     ||nodes[nodes[cur].next[c]].count==0) return 0;
   cur=nodes[cur].next[c];
  }
  return nodes[cur].count;
 }
};
```

Hashing.h

Description: Self-explanatory methods for string hashing.

```
// Arithmetic mod 2^64-1. 2x slower than mod 2^64 and more
// code, but works on evil test data (e.g. Thue-Morse, where
// ABBA... and BAAB... of length 2^10 hash the same mod 2^64).
// "typedef ull H;" instead if you think test data is random,
// or work mod 10^9+7 if the Birthday paradox is not a problem.
typedef uint64_t ull;
struct H {
  ull x; H(ull x=0) : x(x) {}
  H operator+(H o) { return x + o.x + (x + o.x < x); }
  H operator-(H o) { return *this + ~o.x; }
  H operator*(H o) { auto m = (__uint128_t)x * o.x;
    return H((ull)m) + (ull)(m >> 64); }
  ull get() const { return x + !~x; }
  bool operator==(H o) const { return get() == o.get(); }
  bool operator<(H o) const { return get() < o.get(); }
};
static const H C = (ll)1e11+3; // (order ~ 3e9; random also ok)

struct HashInterval {
  vector<H> ha, pw;
  HashInterval(string& str) : ha(sz(str)+1), pw(ha) {
    pw[0] = 1;
    rep(i,0,sz(str))
      ha[i+1] = ha[i] * C + str[i],
      pw[i+1] = pw[i] * C;
  }
  H hashInterval(int a, int b) { // hash [a, b)
    return ha[b] - ha[a] * pw[b - a];
  }
};

vector<H> getHashes(string& str, int length) {
  if (sz(str) < length) return {};
  H h = 0, pw = 1;
  rep(i,0,length)
    h = h * C + str[i], pw = pw * C;
  vector<H> ret = {h};
  rep(i,length,sz(str)) {
    ret.push_back(h = h * C + str[i] - pw * str[i-length]);
  }
  return ret;
}

H hashString(string& s){H h{}; for(char c:s) h=h*C+c;return h;}
```

HashTree.h

```
mt19937 rng(chrono::steady_clock::now().time_since_epoch().
    count());
const ll M = 991831889;
const ll C = uniform_int_distribution<ll>(0.1 * M, 0.9 * M)(rng
    );

struct HashTree {
    struct Hash {
        ll hash = 0;
        ll mul = 1;
        Hash operator+(Hash other) {
            return {(hash * other.mul + other.hash) % M, mul *
                 other.mul % M};
        }
    };
    int n; vector<Hash> tree;

    HashTree(string s) {
        n = 1;
        while (n < s.size()) n *= 2;
        tree.resize(2 * n);
        for (int i = 0; i < s.size(); i++) {
            tree[n + i] = {s[i], C};
        }
        for (int i = n - 1; i >= 1; i--) {
            tree[i] = tree[2 * i] + tree[2 * i + 1];
        }
    }
    void set(int k, char x) {
        k += n;
        tree[k] = {x, C};
        for (k /= 2; k >= 1; k /= 2) {
            tree[k] = tree[2 * k] + tree[2 * k + 1];
        }
    }
    ll hash(int l, int r) {
        l += n;
        r += n;
        Hash left, right;
        while (l <= r) {
            if (l % 2 == 1) left = left + tree[l++];
            if (r % 2 == 0) right = tree[r--] + right;
            l /= 2; r /= 2;
        }
```

```
        return (left + right).hash;
    }
};
```

SuffixArray.h

Description: Builds suffix array for a string. `sa[i]` is the starting index of the suffix which is i'th in the sorted suffix array. The returned vector is of size $n+1$, and `sa[0] = n`. The `lcp` array contains longest common prefixes for neighbouring strings in the suffix array: `lcp[i] = lcp(sa[i], sa[i-1])`, `lcp[0] = 0`. The input string must not contain any nul chars.
Time: $\mathcal{O}(n \log n)$

```
struct SuffixArray {
  vi sa, lcp;
  SuffixArray(string s, int lim=256) { // or vector<int>
    s.push_back(0); int n = sz(s), k = 0, a, b;
    vi x(all(s)), y(n), ws(max(n, lim));
    sa = lcp = y, iota(all(sa), 0);
    for (int j = 0, p = 0; p < n; j = max(1ll, j * 2), lim = p)
        {
      p = j, iota(all(y), n - j);
      rep(i,0,n) if (sa[i] >= j) y[p++] = sa[i] - j;
      fill(all(ws), 0);
      rep(i,0,n) ws[x[i]]++;
      rep(i,1,lim) ws[i] += ws[i - 1];
      for (int i = n; i--;) sa[--ws[x[y[i]]]] = y[i];
      swap(x, y), p = 1, x[sa[0]] = 0;
      rep(i,1,n) a = sa[i - 1], b = sa[i], x[b] =
        (y[a] == y[b] && y[a + j] == y[b + j]) ? p - 1 : p++;
    }
    for (int i = 0, j; i < n - 1; lcp[x[i++]] = k)
      for (k && k--, j = sa[x[i] - 1];
          s[i + k] == s[j + k]; k++);
  }
};
```

AhoCorasick.h

```
struct Aho{
 static const int K=26; // alphabet size
 struct Vertex{
    int next[K],link,term_link; // suffix link, terminal link
    bool output; // true if this node is end of some pattern
    vector<int> ids; // pattern ids ending here
    int len;
    Vertex(){
      fill(begin(next), end(next), -1);
      link=-1;  term_link=-1;  output=false;  len=0;
    }
 };
 vector<Vertex> t = {Vertex()}; // root = state 0
 void add_string(const string& s,int id){
  int v=0;
  for(char ch:s){
   int c=ch-'a';
   if(t[v].next[c] == -1){
     t[v].next[c] = t.size();  t.emplace_back();
   }
   v=t[v].next[c];
  }
  t[v].output=true; t[v].ids.push_back(id);
 }
 void build_automaton(){ // build the automaton T.T
  queue<int> q;
  t[0].link=0;  t[0].term_link=0;
  for(int c=0;  c<K;  c++){
   int u=t[0].next[c];
   if(u!=-1){ t[u].link=0; q.push(u); t[u].len=1; }
   else t[0].next[c]=0;// missing edge from root loops to root
  }
  while(!q.empty()){
   int v=q.front(); q.pop();
   for(int c=0;  c<K;  c++){
    int u=t[v].next[c];
    if(u!=-1){ // compute suffix link
     t[u].link = t[t[v].link].next[c];
     t[u].len = t[v].len+1;
     q.push(u);
    }
    else t[v].next[c] = t[t[v].link].next[c];
   }
   // compute terminal link
   if(t[t[v].link].output) t[v].term_link = t[v].link;
   else t[v].term_link = t[t[v].link].term_link;
  }
 }
 void form_tree(){
    gr.resize(t.size());
    L(state,1,t.size()-1){
     gr[t[state].term_link].push_back(state);
    }
  }
};
```

Eertree.h

```
sz - 2 // no. of distinct palindromes
no. of unique palindromes is linear !!! may use with hashing

use cnt during construction. After extend(i):
last = node representing the longest palindrome ending at pos i
pt.t[pt.last].cnt; // no. of pals ending here, smaller included

use oc after calc.
t[i].oc //no. of occurence of this exact pal[smaller excluded]
sum(t[i].oc) // total no. of palindromic substrings

struct PalindromicTree {
  struct node {
    int nxt[26]; // transitions (adding char to both ends)
    int len;     // length of palindrome
    int st, en;  // start and end index in original string
    int link;    // suffix link (largest palindromic suffix)
    ll cnt;      // no. of palindromic suffixes ending here
    ll oc;       // no. of occurrences of this palindrome
  };
  string s;
  vector<node> t;
  int sz, last;
  PalindromicTree() {}
  PalindromicTree(string _s) {
    s = _s;
    int n = s.size();
    t.clear();
    t.resize(n + 9);
    sz = 2, last = 2;
    t[1].len = -1, t[1].link = 1;
    t[2].len = 0, t[2].link = 1;
  }
 int extend(int pos){//returns 1 if it creates a new palindrome
    int cur = last, curlen = 0;
    int ch = s[pos] - 'a';
    while (1) {
      curlen = t[cur].len;
      if(pos-1-curlen >= 0 && s[pos-1-curlen] == s[pos]) break;
      cur = t[cur].link;
    }
    if (t[cur].nxt[ch]) {
      last = t[cur].nxt[ch];
      t[last].oc++;
      return 0;
    }
    sz++;
    last = sz;
    t[sz].oc = 1;
    t[sz].len = t[cur].len + 2;
    t[cur].nxt[ch] = sz;
    t[sz].en = pos;
    t[sz].st = pos - t[sz].len + 1;
    if (t[sz].len == 1) {
      t[sz].link = 2;
      t[sz].cnt = 1;
      return 1;
    }
    while (1) {
      cur = t[cur].link;
      curlen = t[cur].len;
      if (pos-1-curlen >= 0 && s[pos-1-curlen] == s[pos]) {
        t[sz].link = t[cur].nxt[ch];
        break;
      }
    }
    t[sz].cnt = 1 + t[t[sz].link].cnt;
    return 1;
  }
  void calc_occurrences() {
    for (int i = sz; i >= 3; i--) t[t[i].link].oc += t[i].oc;
  }
};
```

SuffixAutomaton.h

```
/*
 * 1. 1-to-1 Match: Paths starting from root (state 0)
      correspond 1-to-1 with all distinct substrings of S.
 * 2. Equivalence Class: Each state represents a set of
      substrings sharing the exact same endpos set.
 * 3. Length Range: Substrings in state u form a contiguous
      length interval [len(link(u)) + 1, len(u)].
 * 4. Suffix Link: Points to the state containing the longest
      suffix of u with a larger endpos set (forms a tree).
 * 5. Transitions: Transition u --c--> v appends 'c' to all
      strings in u into state v (len(v) >= len(u) + 1).
 * 6. Terminals: States containing suffixes of S (found by
      following suffix links from the final state).
 */
struct SAM{
 struct State{
  int link, maxlen;
  map<char,int> next;
  State(){ link=-1; maxlen=0; }
 };
 vector<State> st;
 vector<vector<int>> gr;
 vector<ll> koybar;
 vector<bool> was_terminal;
 int last,sz;
 ll total; // number of unique substrings
 SAM(int n){
  st.resize(2*n);  gr.resize(2*n);  koybar.resize(2*n);
       was_terminal.resize(2*n);
  st[0]=State();  sz=1;  last=0;  total=0;
 }
 void extend(char c){
  int cur = sz++;
  st[cur].maxlen = st[last].maxlen+1;
  int p=last;
```

```
  while(p!=-1 && !st[p].next.count(c)){
   st[p].next[c]=cur;  p=st[p].link;
  }
  if(p == -1){ st[cur].link=0; }
  else{
   int q = st[p].next[c];
   if(st[p].maxlen+1 == st[q].maxlen){ st[cur].link=q; }
   else{
      int clone=sz++;
      st[clone] = st[q];
      st[clone].maxlen = st[p].maxlen + 1;
      while(p!=-1 && st[p].next[c] == q){
        st[p].next[c]=clone;  p=st[p].link;
      }
      st[q].link = st[cur].link = clone;
    }
  }
    last=cur; was_terminal[last]=true;
    total+=st[cur].maxlen-st[st[cur].link].maxlen;
 }
 void build_tree(){ // USE (sam.sz-1) when you need to visit
      all states
  L(i,1,sz-1) gr[st[i].link].push_back(i);
 }
 void guno(){ // count repetitions in the string for each node
  int cur=last;
  L(i,1,sz-1) koybar[i] = was_terminal[i];
  function<void(int)> dfs=[&](int node){
   for(int ch:gr[node])
      { dfs(ch); koybar[node] += koybar[ch]; }
  };
  dfs(0);
 }
 ll koyta(int i){ return st[i].maxlen-st[st[i].link].maxlen; }
};
```

MinRotation.h

Description: Finds the lexicographically smallest rotation of a string.
Usage: `rotate(v.begin(), v.begin()+minRotation(v), v.end());`
Time: $\mathcal{O}(N)$

```
int minRotation(string s) {
  int a=0, N=sz(s); s += s;
  rep(b,0,N) rep(k,0,N) {
    if (a+k == b || s[a+k] < s[b+k]) {b += max(0, k-1); break;}
    if (s[a+k] > s[b+k]) { a = b; break; }
  }
  return a;
}
```

BitTrie.h

```
struct BitTrie{
 struct Node{
    int child[2],cnt;//trie[0]=total numbers inserted in trie
    Node(){ child[0] = child[1] = -1;  cnt=0; }
  };
 vector<Node> trie;
 int BITS;

 BitTrie(int maxBits = 30){ // 30 for numbers up to 1e9
  BITS = maxBits;  trie.push_back(Node()); // root
}

 void insert(int x){
  int node=0;
  trie[node].cnt++;
  for(int b=BITS;  b>=0;  b--){
   int bit = (x>>b)&1;
   if(trie[node].child[bit] == -1){
    trie[node].child[bit] = trie.size();
    trie.push_back(Node());
   }
   node = trie[node].child[bit];  trie[node].cnt++;
   }
 }

 void erase(int x){
  int node=0;
  trie[node].cnt--;
  for(int b=BITS;  b>=0;  b--){
   int bit = (x>>b)&1;
   int nxt=trie[node].child[bit];
   node=nxt; trie[node].cnt--;
  }
 }

 ll max_xor(ll x){ // find y from this trie that maximizes x^y
  if(trie[0].cnt == 0) return 0; // empty trie
  ll cur=0, y=0;
  R(bit, BITS, 0){
   int nextbit = getBit(x,bit); // at root, nextbit=30
   if(trie[cur].child[!nextbit] != -1
    and trie[trie[cur].child[!nextbit]].cnt){
    cur = trie[cur].child[!nextbit];
    if(!nextbit) y = setbit(y,bit);
  }
   else{ // has to exist, since tire is not empty
    cur = trie[cur].child[nextbit];
    if(nextbit) y = setbit(y,bit);
   }
  }
  return y; //return x^y if you need the xor
 }
};
```

Various (9)

TernarySearch.h

Description: Find the smallest i in $[a, b]$ that maximizes $f(i)$, assuming that $f(a) < \ldots < f(i) \geq \cdots \geq f(b)$. To reverse which of the sides allows non-strict inequalities, change the $<$ marked with (A) to $<=$, and reverse the loop at (B). To minimize f, change it to $>$, also at (B).
Usage: `int ind = ternSearch(0,n-1,[&](int i){return a[i];});`
Time: $\mathcal{O}(\log(b-a))$

```
template<class F>
int ternSearch(int a, int b, F f) {
  assert(a <= b);
  while (b - a >= 5) {
    int mid = (a + b) / 2;
    if (f(mid) < f(mid+1)) a = mid; // (A)
    else b = mid+1;
  }
  rep(i,a+1,b+1) if (f(a) < f(i)) a = i; // (B)
  return a;
}
```

9.1 Bitwise Identities

Some properties of bitwise operations

- $a|b = a \oplus b + a\&b$
- $a \oplus (a\&b) = (a|b) \oplus b$
- $(a\&b) \oplus (a|b) = a \oplus b$

Addition

- $a + b = a|b + a\&b$
- $a + b = a \oplus b + 2(a\&b)$

submasks.h

```
for (int mask = 0; mask < (1<<n); mask++) {
    for(int i = mask;  ;  i = (i-1) & mask) {
        // ...
        if(i == 0) break;
    }
}
```

int128.h

```
using i128 = __int128_t;

i128 read() {
  string s; cin >> s;
  bool neg = false; size_t i = 0;
  if (s[0] == '-') { neg = true; i = 1; }
  i128 mag = 0;
  for (; i < s.size(); ++i) mag = mag * 10 + (s[i] - '0');
  return neg ? -mag : mag;
}

void print(i128 x) {
  if (x == 0) { cout << 0 << nl; return; }
  bool neg = x < 0;
  if (neg) x = -x;
  string s;
  while (x) {
    s.push_back(char('0' + (int)(x % 10)));
    x /= 10;
  }
  if (neg) s.push_back('-');
  reverse(s.begin(), s.end());
  cout << s << nl;
}

bool cmp(i128 x, i128 y) { return x > y; } // for sorting
```

FloorCeilDiv.h

```
ll floor_div(ll a, ll b) {
    if (b < 0) a = -a, b = -b;
    if (a >= 0) return a / b;
    return (a - b + 1) / b;
}
ll ceil_div(ll a, ll b) {
    if (b < 0) a = -a, b = -b;
    if (a >= 0) return (a + b - 1) / b;
    return a / b;
}
```